I

Name _____

Roll No. _____ Year 20____20____

Exam Seat No. _____

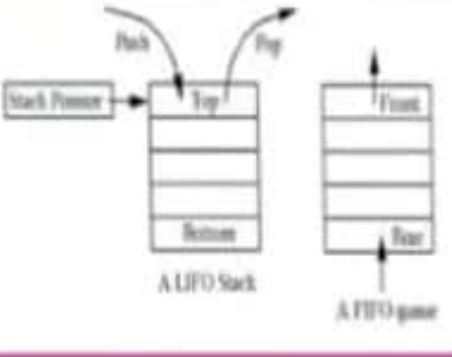
COMPUTER GROUP | SEMESTER - III | DIPLOMA IN ENGINEERING AND TECHNOLOGY

A LABORATORY MANUAL

FOR

DATA STRUCTURE USING 'C'

(22317)



MAHARASHTRA STATE BOARD OF TECHNICAL EDUCATION, MUMBAI

(Autonomous) (ISO 9001 : 2015) (ISO / IEC 27001 : 2013)

Practical No. 1: Program to Perform Operations on Array

I. Practical Significance:

Arrays are used to implement other data structures, such as list, queues, stacks, strings, hash, etc. By using Array student should be able to create, insert, delete and display the contents from array.

II. Relevant Program Outcomes (POs)

- **Basic knowledge:** Apply knowledge of basic mathematics, sciences and basic engineering to solve the computer group related problems.
- **Discipline knowledge:** Apply Information Technology knowledge to solve broad-based Information Technology related problems.
- **Experiments and practice:** Plan to perform experiments and practices to use the results to solve the computer group related problems.
- **Engineering tools:** Apply relevant Computer programming / technologies and tools with an understanding of the limitations.
- **Communication:** Communicate effectively in oral and written form.

III. Competency and Practical skills

This practical expects to develop the following skills in the student.

Develop 'C' programs to solve computer group related problems.

1. Write algorithm and draw flow chart for Array.
2. Write/Compile/Debug and Save C program for operations on array.

IV. Relevant Course Outcome(s)

- Perform basic operations on array.

V. Practical Outcome (PrOs)

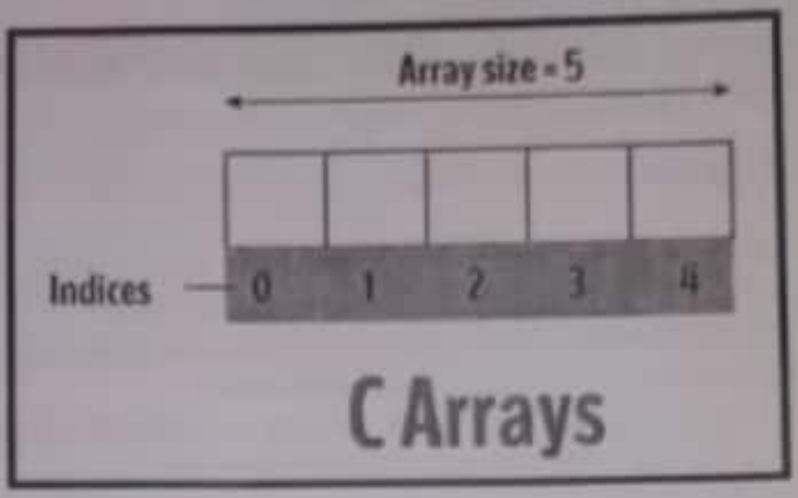
- Implement a 'C' program to perform following operations on Array : Creation, Insertion, Deletion, Display.

VI. Relevant Affective domain related Outcome(s)

1. Follow safety measures.
2. Follow ethical practices.

VII. Minimum Theoretical Background

Array:



An array is a collection of data that holds fixed number of values of same type. For example: if you want to store marks of 5 students, you can create an array for it.

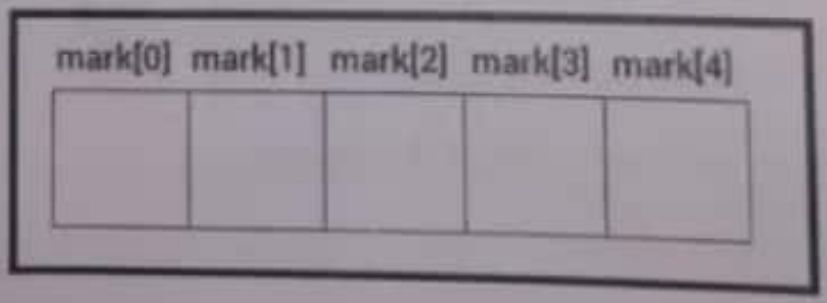
Declaration of an array in C:

Syntax: data_type array_name [array_size];

Ex. : int mark[5];

Array elements are accessed by indices.

Suppose an array *mark [5] declared* as above. The first element is *mark [0]*, second element is *mark [1]* and so on.



- Arrays have 0 as the first index not 1. In this example, *mark[0]*
- If the size of an array is *n*, then to access the last element, (n-1) index is used. In this example, *mark[4]*
- Suppose the starting address of mark [0] is 2120. Then, the next address, mark [1], will be 2122, address of mark [2] will be 2124 and so on. It's because the size of a int is 2 bytes.

VIII. Algorithm:

1. Start
2. Set J = K
3. Repeat steps 4 and 5 while J < N
4. Set LA [J] = LA [J + 1]
5. Set J = J + 1
6. Set N = N - 1
7. Stop

IX. Flowchart:

X. C Program Code:

```
#include <stdio.h>
#include <conio.h>
void main()
{
int a[10], n, i, n, pos;
clrscr();
printf("\n Enter the number of array element\n");
scanf("\n %d", &n);
printf("\n Enter the array with %d element\n", n);
for(i=0; i<n; i++)
scanf("\n %d", &a[i]);
printf("\n Enter the key value and its position\n");
scanf("\n\n %d %d", &x, &pos);
for(i=n; i>=pos; i--)
{
a[i] = a[i-1];
}
a[pos-1] = x;
printf("\n Array element\n")
for(i=0; i<n+1; i++)
printf("\n %d\t", a[i]);
getch();
}
```

XI. Resources required

Sr. No.	Name of Resource	Specification	Quantity	Remarks
1	Hardware: Computer System	Computer (i3-i5 preferable), RAM minimum 2 GB and onwards	As per batch size	For all Experiments
2	Operating system	Windows 7 or Later Version/LINUX version 5.0 or Later Version		
3	Software	Turbo C /C++ Version 3.0 or Later Version		

XII. Precautions

1. Array should be declared and accessed properly.
2. Save the program in specific directory / folder.
3. Follow safety practices.

XIII. Resources used

S. No.	Name of Resource	Specification
1	Computer System with broad specifications	computer (i5-i5 preferable) RAM minimum 2 GB and onwards
2	Software	Turbo c/c++ version 3.0 or later version
3	Any other resource used	Windows 7 or Later version / LINUX version or later version

XIV. Results (Output of the Program)

Enter number of array element 5
Enter array with 5 element 2 4 10 5 7
Enter the key value and its position 9 5
Array element 2 4 10 5 9 7

XV. Conclusion(s)

We have perform the practical of operations on array successfully.

XVI. Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

1. Consider array size as 10. There are 8 elements in the array. The value (say 100) is to be inserted at index 9. What will be the output?
2. Consider array size as 10. There are 6 elements in the array. The value at index 7 is to be deleted. What will happen?

(Space for answers)

① consider array
A[10] = {10, 20, 30, 40, 50, 60, 70, 80};
A[9] = 100;
The output will be;
values of array:
10, 20 30 40 50 60 70 80 100

(2) Consider array

A [10] = {15, 25, 35, 45, 55, 65};

There will be a error that there is no value at pos 7 or else;

The output will be:

values of array:

15 25 35 45 55 65

XVII. Exercise

1. Perform the given operations on the following array and show diagrammatic presentation of each operation given below:

0	1	2	3	4	5	6	7	8	9	10
10	20	30	40	50	60	70	80	90		

a) Delete element at index 8
b) Add element at index 4 with value 999
c) Delete element at index 0

2. Insert Array2 at index 2 of Array1 :

Array1:

0	1	2	3	4	5	6	7	8	9	10
10	20	30	40	50						

Array2:

0	1	2
100	200	300

(Space for answers)

1. a)

0	1	2	3	4	5	6	7	8	9	10
10	20	30	40	50	60	70	80			

b)

0	1	2	3	4	5	6	7	8	9	10
10	20	30	40	999	50	70	80			

c)

0	1	2	3	4	5	6	7	8
NULL	20	30	40	50	60	70	80	

Practical No. 2: Search a Data Using Linear Search

I. Practical Significance

In order to perform some operation on specific data element, the data has to be searched in data collection, and the system requires following a searching method. One of the most commonly used method is linear search for searching data from the given list.

II. Relevant Program Outcomes (POs)

- **Basic knowledge:** Apply knowledge of basic mathematics, sciences and basic engineering to solve the broad-based Computer engineering problem.
- **Discipline knowledge:** Apply Information Technology knowledge to solve broad-based Information Technology related problems.
- **Experiments and practice:** Plan to perform experiments, practices and to use the results to solve Information Technology related problems.
- **Engineering tools:** Apply appropriate Computer Engineering / Information Technology related techniques/ tools with an understanding of the limitations.
- **Communication:** Communicate effectively in oral and written form.

III. Competency and Practical skills

This practical is expect to develop the following skills in you

Develop 'C' programs to solve broad-based computer group related problems.

1. Write algorithm and draw Flow Chart for Linear Search.
2. Write/Compile/Debug and Save C program for function to search using Linear Search.

IV. Relevant Course Outcome(s)

- Apply different searching and sorting techniques.

V. Practical Outcome (PrOs)

Implement a 'C' program to search a particular data from the given Array using Linear Search.

VI. Relevant Affective domain related Outcome(s)

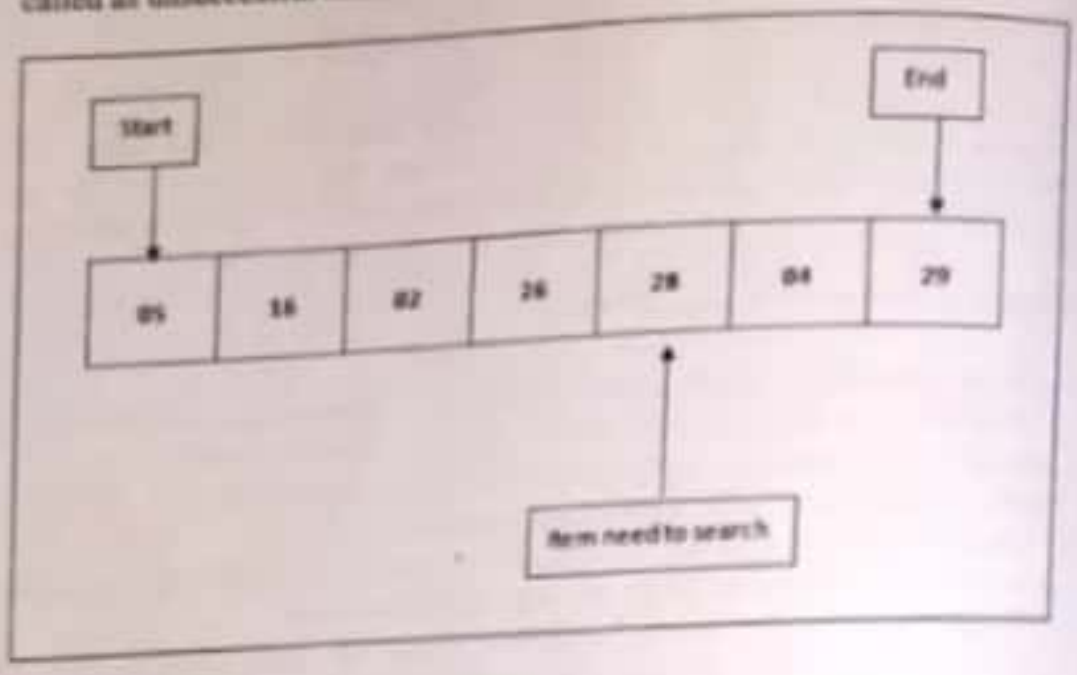
1. Follow safety measures
2. Follow ethical practices.

VII. Minimum Theoretical Background

Searching: Searching is just finding out the data item among given list. Two cases are possible that either item is found or item not present in the list.

Linear search: Linear Search is a very simple search algorithm. In this sequential search is done by searching items one by one. Each item is checked and if a match is found then that particular item is returned, otherwise the search is continues till end of

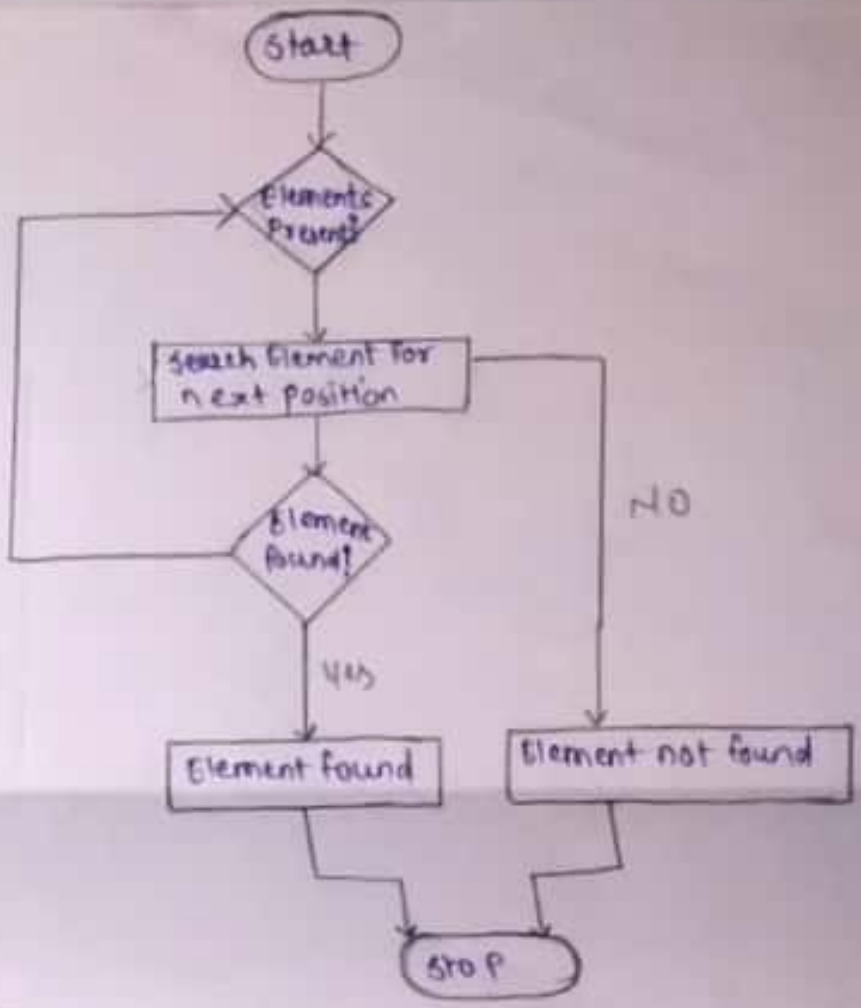
the list of items. If item which is to be searched is not present in list then the search is called as unsuccessful search.



VIII. Algorithm

1. Set j to 1
2. If j>n, jump to step 7.
3. If x[] == i, jump to step 6.
4. Then, increment j by 1 i.e. j=j+1
5. Go back to step2.
6. Display the element i which is found at particular index f, then jump to step 8.
7. Display element not found in the set of input element
8. Exit /End.

IX. Flow Chart




```
{
int a[20],i,x,n;
clrscr();
printf ("\n How many element ? ");
scanf ("\n %d ", &n);
printf ("\n\n Enter array elements:\n");
for (i=0; i<n; ++i)
scanf ("\n\n %d", &a[i]);
printF ("\n\n Enter element to search: ");
scanf ("\n %d" &x);
for (i=0; i<n; ++i)
if (a[i]==x)
 break;
```



```
if (i<n)
   print f ("\n\n Element found at index %d",i);

else
   printf ("\n\n Element not found");
```

X. 'C' Program Code

```
#include <stdio.h>
#include <conio.h>
int main()
{
 int a[20],i,x,n;
 clrscr();
 printf ("\n How many element ? ");
 scanf ( "\n %d ", &n);
printf ("\n\n Enter array elements:\n");
for (i=0 ; i<n ; ++i)
scanf ("\n\n %d", &a[i]);
printf ("\n\n Enter element to search: ");
scanf ("\n %d" &x);
for (i=0 ; i<n ; ++i)
 if (a[i] == x)
  break;
```



```
 if (i<n)
    print f ("\n\n Element found at index %d", i);

 else
  printf ("\n\n Element not found");
 return 0;
getch();
}
```

XI. Resources required

Sr. No.	Name of Resource	Specification	Quantity	Remarks
1	Hardware: Computer System	Computer (i3-i5 preferable), RAM minimum 2 GB onwards	As per batch size	For all Experiments
2	Operating system	Windows 7 or Later Version/LINUX version 5.0 or Later Version		
3	Software	Turbo C /C++ Version 3.0 or Later Version		

XII. Precautions

1. Save the program in specific directory / folder.
2. Follow safety practices.

XIII Resources used

S. No.	Name of Resource	Specification
1	Computer System with broad specifications	Computer(i3-i5 preferable) RAM minimum 2 GB onwards
2	Software	Turbo c/c++ version 3.0 or later version
3	Any other resource used	Windows 7 or later version /LINUX version 5.0 or later version

XIV Result (Output of the Program)

How many elements? 6
Enter array elements: 10 15 20 25 35 45
Enter elements to search: 20
Element found at index 2

XV Conclusion(s)

We have perform this practical of search a data using linear search successfully.

XVI Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

1. Linear Search is most suitable for which kind of data list?
2. What is the output if list of item having same data item which is to be searched?

(Space for answers)

① It is practical to implement linear search in the situations mentioned in when the list has only a few elements and when performing a single search in unordered list, but for larger elements the complexity becomes larger and it makes sense to sort the list and employ binary search or hashing.

② The output if list of item having same data item is to be searched is based on its order and preference.

Explanation: If list of item contain 5 items in which two are same and other are different.

XVII Exercise

1. Consider following list to perform Linear Search.
   56, 36, 89, 56, 01, 00, 67, 59
   i. Search the item 01 from above list and write the item is found or not with procedure.
   ii. Search the item 55 from above list. Write the item is found or not with procedure.
2. State the limitations of Linear Search in terms of Time Complexity.

(Space for answers)

② A linear search runs in at worst linear time

and makes at most n comparisions, where 'n' is the length of list. Linear search is rarely practical because other search algorithms and schemes, such as binary search algorithm and hash tables, allow significantly faster searching for all but short lists.

Q7

i) First the value is read, then it is compared to first value 56, 01 is not equal to 56 so now it compares 01 with 36, 36 is not equal to 01 so it continues and reads value at 2nd pos. i.e. 89, 89 != 01, therefore it reads value at 3rd pos i.e. 56, 56 != 01, therefore it reads value at 4th pos i.e. 01. Now, 01 == 01 so it will display the message, "Value present at 04 pos!".

ii) First value is read, then it is compared to first value 56, 55 != 56, so now it compares 55 with 36, 36 ≠ 55 so it continues and reads value at 2nd pos i.e. 89, 89 != 55. ∴ it reads value at 3rd pos i.e. 56, 56 != 55. ∴ it reads value at 4th pos i.e. 01. 55 != 01 ∴ it reads value at 5th pos i.e. 00. 55 != 00 ∴ it reads value at 6th pos is 67. 67 != 01, ∴ it reads value at 7th pos i.e. 59. Now, no value for 55 so it will the message, "Value is not present!"

Practical No. 3: Search a data using binary search

I. Practical Significance

In order to perform some operation on specific data element, the data has to be searched in data collection, and the system requires following a searching method. One of the most commonly used methods is Binary search for searching data from the given list. Binary Search is suitable of large scale of data.

II. Relevant Program Outcomes (POs)

- **Basic knowledge:** Apply knowledge of basic mathematics, sciences and basic engineering to solve the broad-based Computer engineering problem.
- **Discipline knowledge:** Apply Information Technology knowledge to solve broad- based Information Technology related problems.
- **Experiments and practice:** Plan to perform experiments, practices and to use the results to solve Information Technology related problems.
- **Engineering tools:** Apply appropriate Computer Engineering / Information Technology related techniques/ tools with an understanding of the limitations.
- **Communication:** Communicate effectively in oral and written form.

III. Competency and Practical skills

This practical is expect to develop the following skills in student.

Develop 'C' programs to solve broad-based computer group related problems.

1. Write algorithm and draw Flow Chart for Binary Search.
2. Write/Compile/Debug and Save C program for function to search using Binary Search.

IV. Relevant Course Outcome(s)

- Apply different searching and sorting techniques.

V. Practical Outcome (POs)

Implement a 'C' program to search a particular data from the given Array using Binary Search

VI. Relevant Affective domain related Outcome(s)

1. Follow safety measures
2. Follow ethical practices.

II. Minimum Theoretical Background

Searching: Searching is, finding out the data item among list given. Two cases are possible that either item is found or item not present in the list.

Binary Search: Binary Search is a useful and fast search algorithm. This searching method uses divide and conquer principal. Binary search uses sorted list to search an item.

Binary Search looks for a particular item by comparing the middle most item of the collection. If match occurs then the index of item is returned. If middle item is greater than the item, the item is searched in the sub-array to the left of the middle item. Otherwise, the item is searched for in the sub-array to the right array as well until the size of the sub-array reduces to zero.

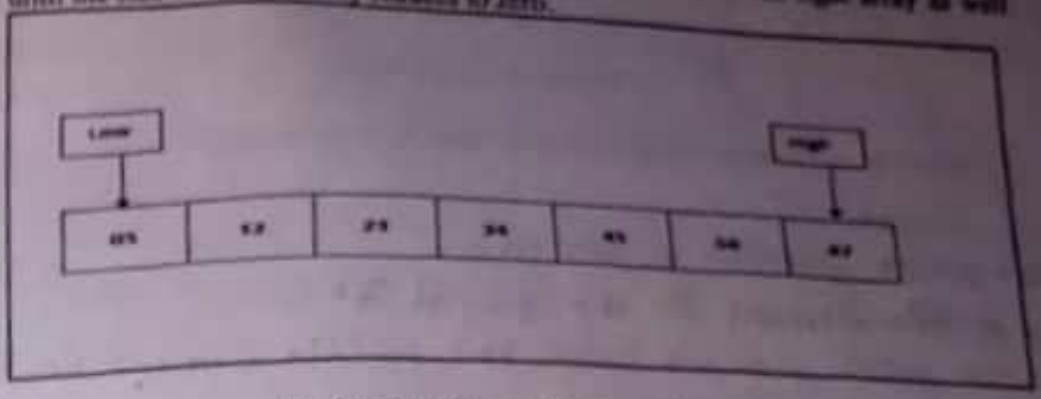


Fig 1.: Sorted array for searching an item

First determine half of the array by using this formula-

Mid=low + (high-low) /2

Here it is, 0+ (6-0)/2= 3. So, 3 is the Mid of the array.

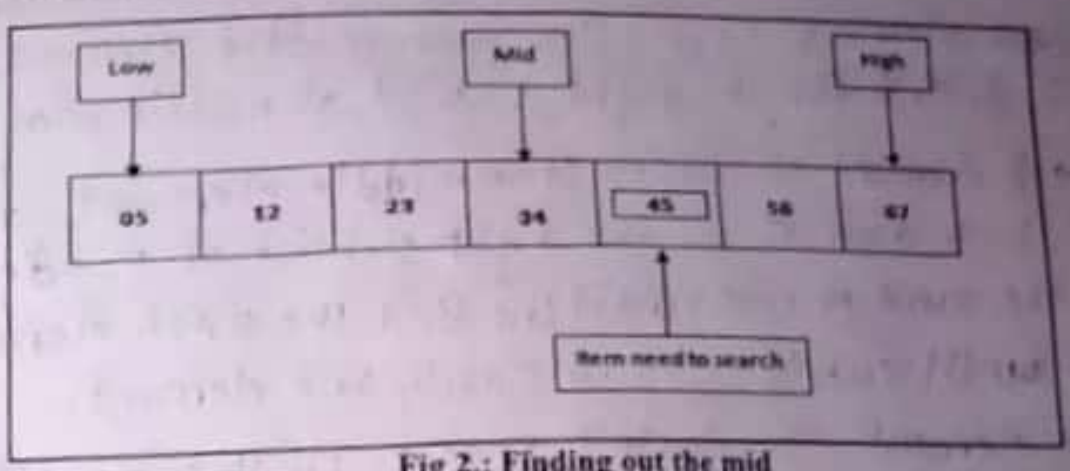


Fig 2.: Finding out the mid

Now we compare the value stored at location 3, with the value being searched, i.e.45. We find that the value at location 3 is 34, which is not a match. As the value is than greater 34 and we have a sorted array, so we also know that the target value must be in the upper portion of the array.

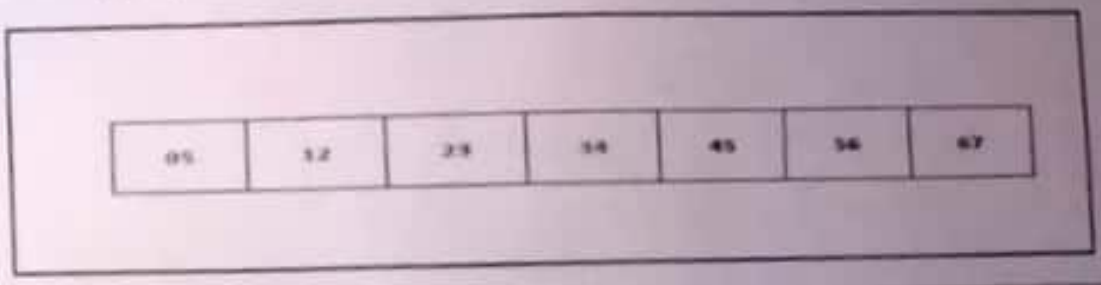


Fig 3. Upper portion of the array

We change our low to mid+1 and find the new mid value again

Low=mid+1

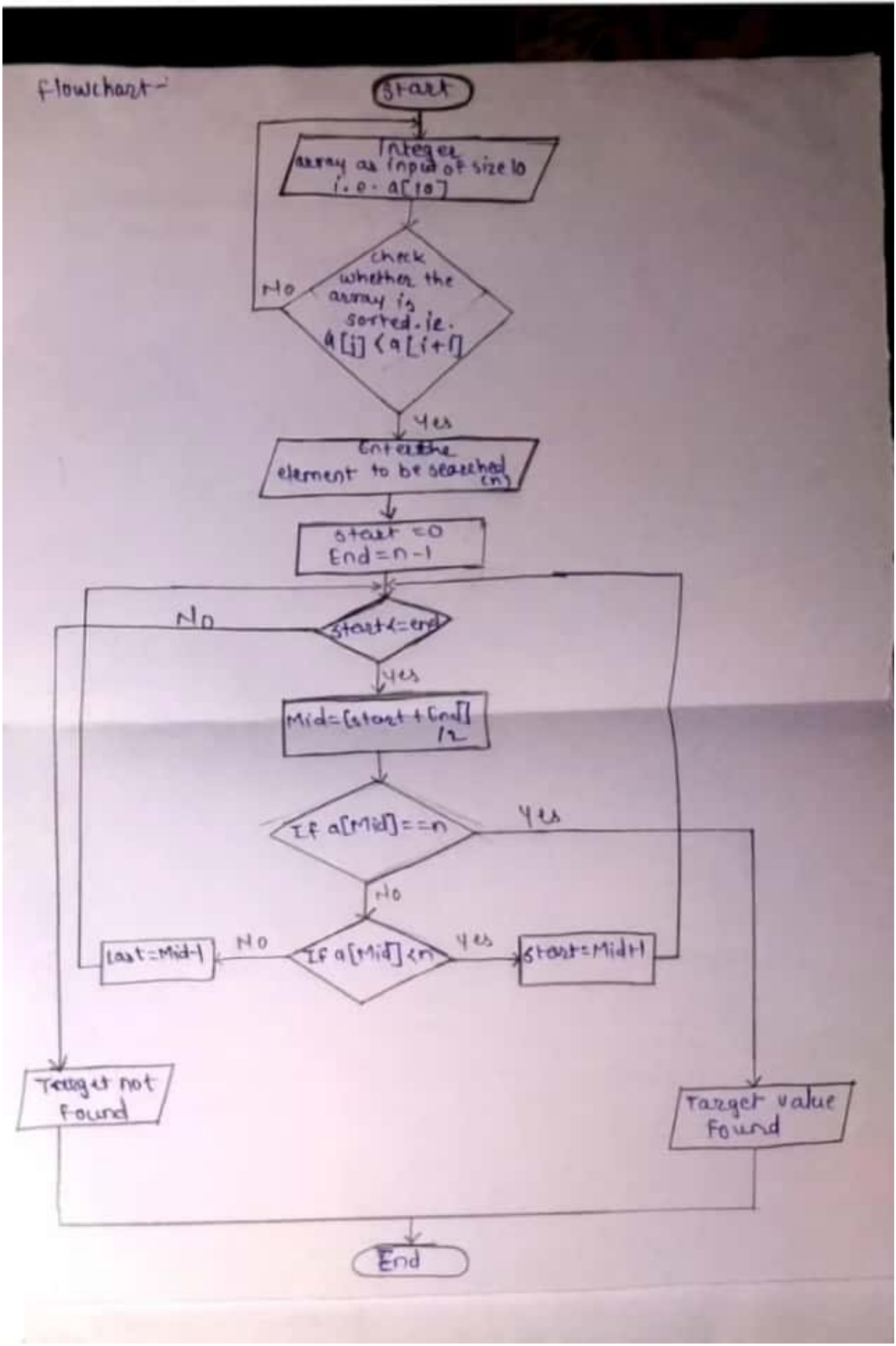
Mid=low + (high-low) /2

Do above given procedure till the item is found if present in the list.

VIII. Algorithm

1. Read the search element from user.
2. Find the middle element in the sorted list.
3. Compare the search element with the middle element in the sorted list.
4. If both are matched, then display "Given element is found !!!" and terminate the function.
5. If both are not matched, then check whether the search element is smaller or larger than the middle element.
6. If search element is smaller than middle element, repeat steps 2, 3, 4, 5 for the right sublist of middle element.
7. If search element is larger than middle element, repeat steps 2, 3, 4 and 5 for the right sublist of middle element.
8. Repeat the same process until we find the search element in list or until sublist contains only one element.
9. If that element also doesn't match with search element, then display "Element is found not in list !!!" and terminate the function.

flowchart-



X. 'C' Program Code

```
#include <stdio.h>
#include <conio.h>
int main()
{
int i, low, high, mid, n, key, array[100];
clrscr();
printf("\n Enter Number of elements:");
scanf("\n %d", &n);
printf("\n Enter %d integers: ", n);
for (i=0; i<n; i++)
scanf("\n %d", &array[i]);
printf("\n Enter value to find:");
scanf("\n %d", &key);
low = 0;
high = n-1;
mid = (low+high)/2;
while (low <= high) {
if (array[mid] < key)
low = mid+1;
else if (array[mid] == key) {
printf("\n %d found at location %d.\n", key, mid+1);
break;
}
else
high = mid-1;
mid = (low+high)/2;
}
if (low > high)
printf("\n Not Found! %d isn't present in the list.\n", key);
return 0;
getch();
}
```

XI. Resources required

Sr. No.	Name of Resource	Specification	Quantity	Remarks
1	Hardware: Computer System	Computer (i3-i5 preferable), RAM minimum 2 GB and onwards	As per batch size	For all Experiments
2	Operating system	Windows 7 or Later Version/LINUX version 5.0 or Later Version		
3	Software	Turbo C /C++ Version 3.0 or Later Version		

XII Precautions

1. Save the program in specific directory / folder.
2. Follow safety practices.

XIII Resources used

S. No.	Name of Resource	Specification
1	Computer System with broad specifications	Computer (i3-i5 preferable), RAM minimum 2 GB and onwards
2	Software	Turbo C/C++ Version 3.0 or later version
3	Any other resource used	Windows 7 or later version/LINUX version 5.0 or later version

XIV Result (Output of the Program)

```
Enter number of elements : 4
Enter 4 integers : 12 45 56 78
Enter value to find : 43
Not found! 43 isn't present in the list.
```

XV Conclusion(s)

We have perform this practical of search a data using binary search successfully.

XVI Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

1. Binary Search is most suitable for what kind of data list.
2. If mid=4.5 then what should be the value of mid?

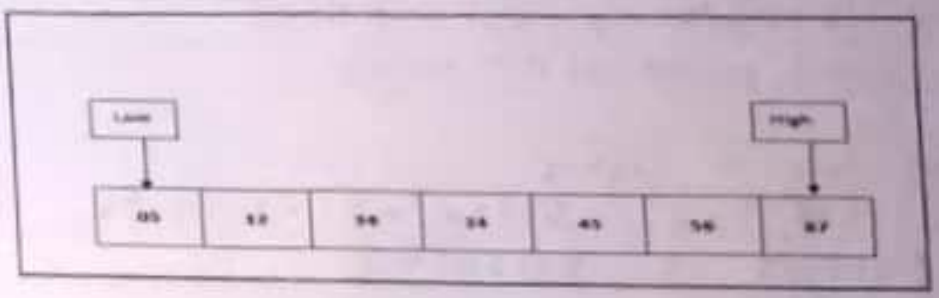
(Space for answers)

(1) Binary search works on sorted arrays. This search begins by comparing an element in the middle of array with the target.

(2) mid = 4.5
midvalue = 14.5

XVII Exercise

1. State the benefits and limitations of Binary Search in terms of Time Complexity.
2. Consider a following list and write down the steps to perform Binary Search to search following numbers.
   i. 12
   ii. 67
   iii. 100



(Space for answers)

① → The input of binary search must be sorted. That being said, the input must be "sortable". So, it cannot just use binary search on list / array of objects. The objects must have "order" to them.
Forex: If you had a list of basket balls and you wanted to find a specific basket balls. You could not just sort them to find the target ball - you would have to analyze all of them. So, a list of basketballs is not a valid input for binary search, but it is for linear search.

② i) 12 : low = 0 , high = 6
mid = (0+6)/2 = 3 , mid = 9
val < a [mid] i.e. 12 < 34
High = mid - 1 / 2 = 2 Mid = (0+2)/2 = 1
A [mid] : 12
val = = a[mid]
∴ value present at 2nd position.

ii) 67 : low = 0, high = 6
Mid = (0+6)/2 = 3
val > a [mid] i.e. 67 > 34
low = mid+1 = 3+1 = 4 mid = (4+6)/2 = 5
A [mid] = 56
val > a [mid] low = mid+1 = 5+1 = 6
Mid = (6+6)/2 = 6 Val = = a [mid]
∴ Value present at 6th position.

iii) 100 : low = 0 , high = 6
Mid = (0+6)/2 = 3 Val > a [mid] i.e. 100 > 34
Mid (4+6)/2 = 5 Val > a [mid]
low = 5+1 = 6
low = 6 , high = 6 mid = 12/2 = 6
Val = = a [mid]
∴ Value is absent.

Practical No. 4: Program to Sort an Array Using Bubble Sort

I. Practical Significance:

In order to perform some operation on specific data element, the data has to be sorted in data collection, and the system requires following a Sorting method. One of the most commonly used methods is Bubble Sort for sorting data from the given list.

II. Relevant Program Outcomes (POs)

- **Basic knowledge:** Apply knowledge of basic mathematics, sciences and basic engineering to solve the computer group related problems.
- **Discipline knowledge:** Apply Information Technology knowledge to solve broad-based Information Technology related problems.
- **Experiments and practice:** Plan to perform experiments and practices to use the results to solve the computer group related problems.
- **Engineering tools:** Apply relevant Computer programming / technologies and tools with an understanding of the limitations.
- **Communication:** Communicate effectively in oral and written form.

III. Competency and Practical skills

This practical expects to develop the following skills in the student.

Develop 'C' programs to solve computer group related problems.

1. Write algorithm and draw flow chart for Bubble sort.
2. Write / Compile / Debug and execute 'C' program for Bubble sort.

IV. Relevant Course Outcome(s)

- Apply different searching and sorting techniques.

V. Practical Outcome (PrOs)

- Implement a 'C' program to sort an array using Bubble Sort method.

VI. Relevant Affective domain related Outcome(s)

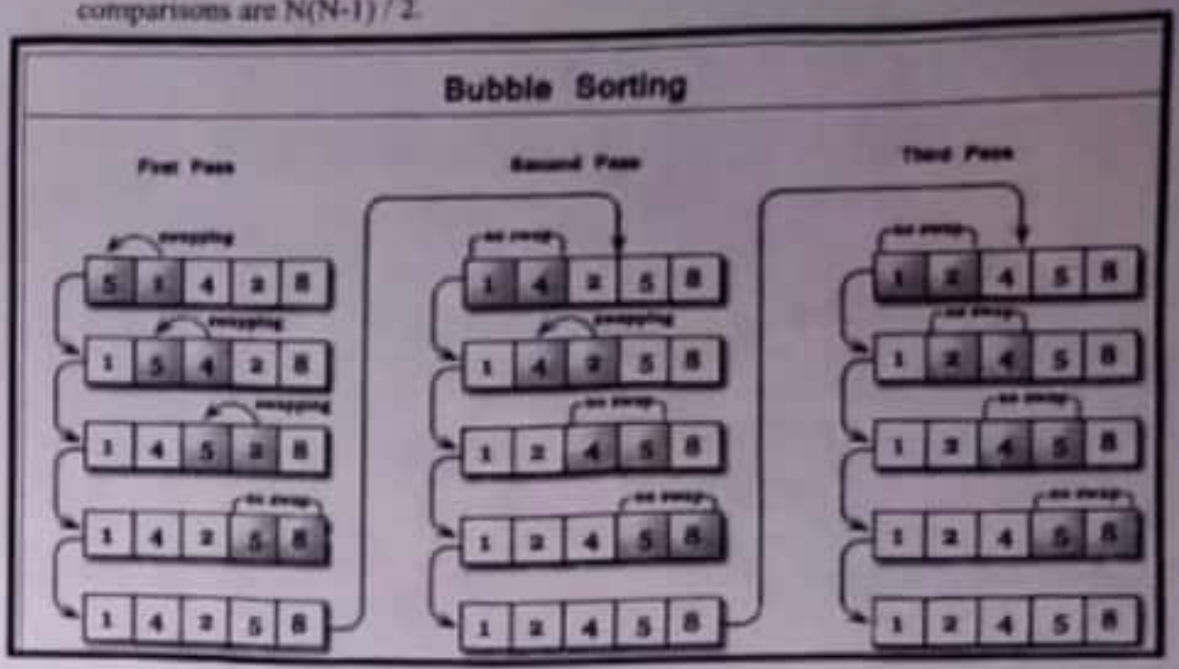
1. Follow safety measures.
2. Follow ethical practices.

VII. Minimum Theoretical Background

In Bubble Sort method the list is rearranged by exchanging the two adjacent elements if they are not in order (order may be ascending or descending).

Bubble sort algorithm starts by comparing the first two elements of an array and swapping if necessary, i.e., if you want to sort the elements of array in ascending order and if the first element is greater than second then, the elements are swapped but, if the first element is smaller than second, elements are not swapped. Then, again second and third elements are compared and swapped if it is necessary and this process continues until last and second last element are compared and swapped. This completes the first iteration of bubble sort.

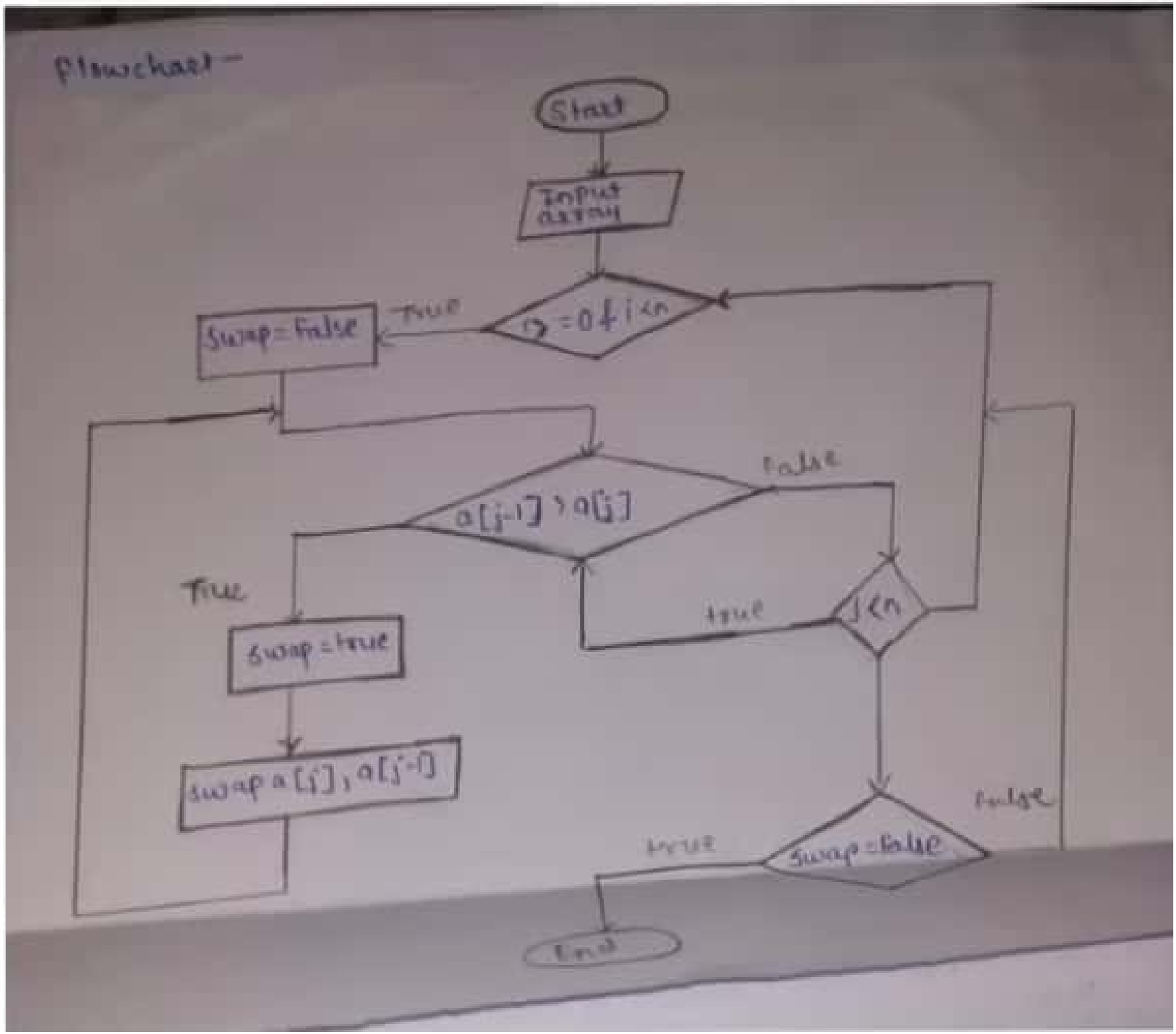
In general it may be required to go through maximum passes as (N-1) for every pass the number of comparisons are (N-1), (N-2), (N-3), ... 1. The maximum number of comparisons are N(N-1) / 2.

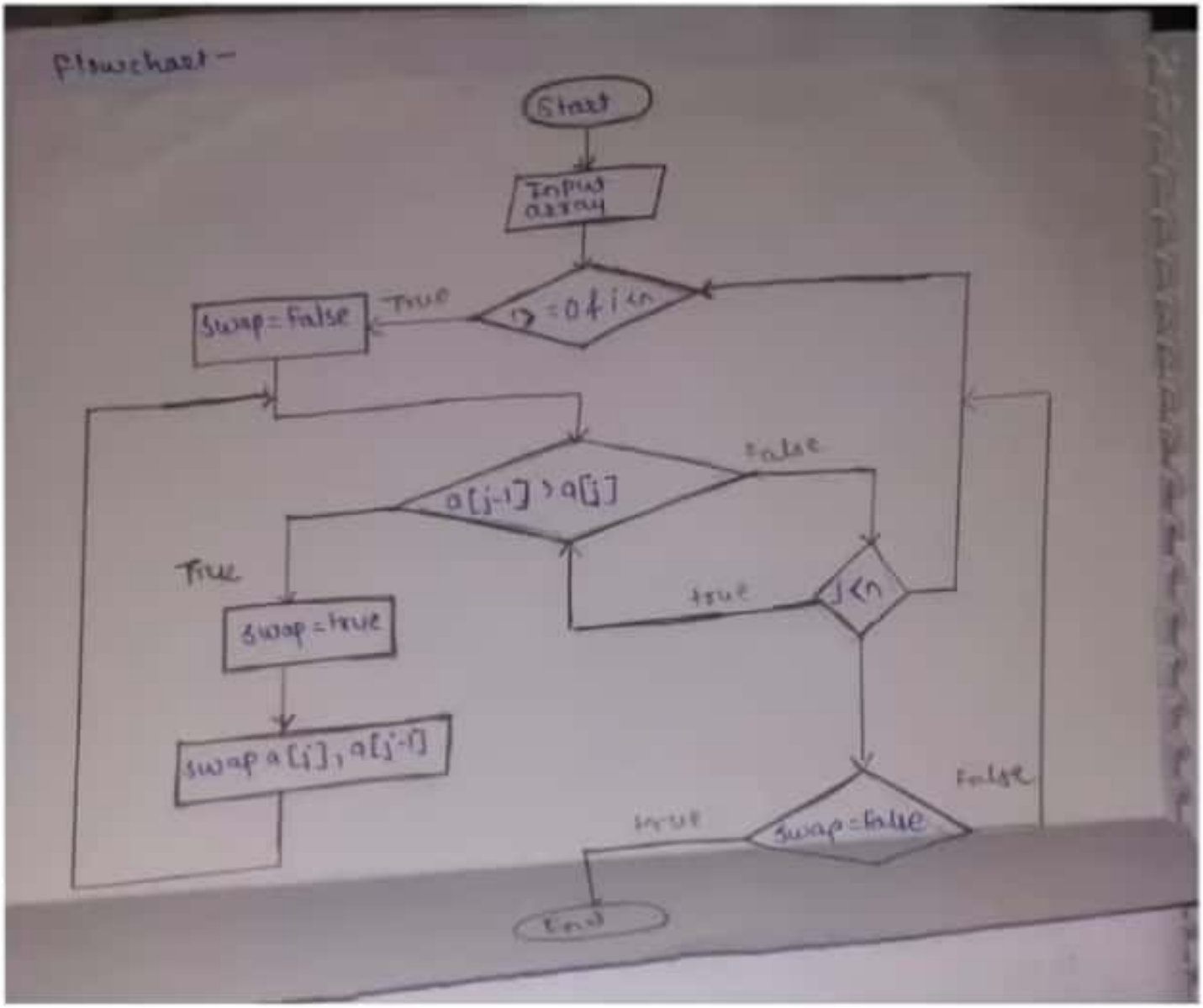


VIII. Algorithm:

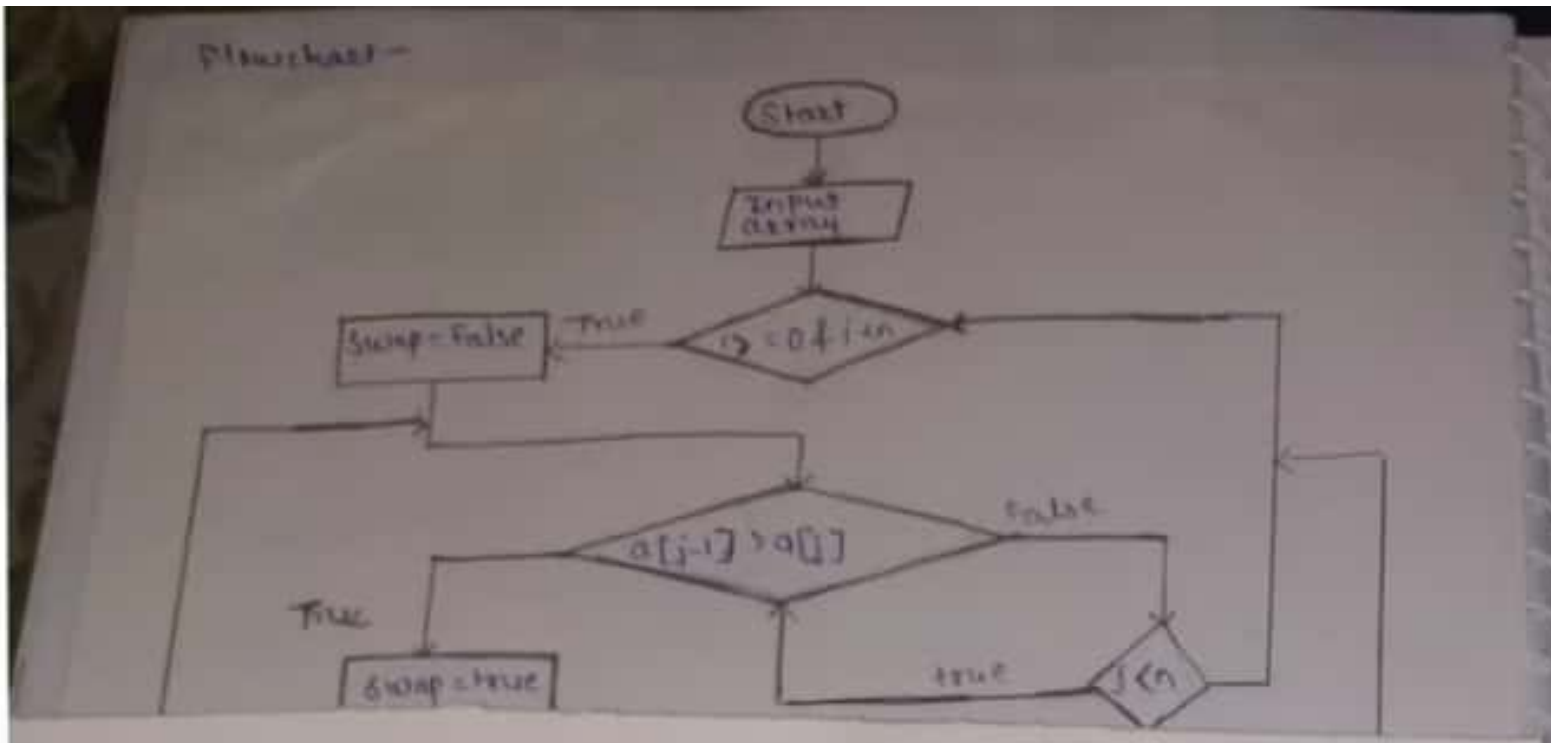
1. Start loop till N no. of elements.
2. Start 2nd loop till N-1 because every time highest element reaches at last position.
3. Compare next two numbers, if they are in wrong order, swap them.
4. Else compare next two elements and repeat the second loop ends and decrement 1st loop.
5. Do this till 1st loop ends.
6. Exit.

Flowchart –





Flowchart –



X. C Program Code:

```
#include <stdio.h>
#include <conio.h>
int main()
{
int a[100], number, i, j, temp;
   clrscr();
printf("\n Please Enter the total Number of Elements: ");
scanf("%d", &number);
printf("\n Please Enter the Array Elements: ");
for (i=0; i<number; i++)
   scanf("%d", &a[i]);
for (i=0; i<number-1; i++)
{
 for (j=0; j<number-i-1; j++)
  {
  if a[j] > a[j+1];
  {
```

```
        temp = a[j];
        a[j] = a[j+1];
        a[j+1] = temp;
      }
    }
  }
  printf("\n List Sorted in Ascending Order :");
  for (i=0; i<number; i++)
  {
    printf("%d\t", a[i]);
  }
  printf("\n");
    return 0;
  getch();
}
```

XIII. Resources used

S. No.	Name of Resource	Specification
1	Computer System with broad specifications	Computer (i3 is preferable) RAM minimum 2GB and above
2	Software	Turbo C/C++ version 3.0 or latter version
3	Any other resources used	Windows 7 or latter version/LINUX version 5.0 or latter version

XIV. Results (Output of the Program)

Please Enter total Number of Elements : 6
Please Enter the Array Element : 12 3 8 0 4 67
List Sorted in Ascending Order : 0 3 4 8 12 67

XV. Conclusion(s)

We have perfor this practicle of program to sort an array with bubble sort successfully.

XVI. Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

1. What is the output of following code?

```
for(i=0;i<=n-1;i++)
{
for(j=0;j<=n-1-i;j++)
{
if(a[j]>a[j+1])
{
temp=a[j];
a[j]=a[j+1];
a[j+1]=temp;
}
}
}
```

2. Find the error(s) (if any) from the following code :

```
for (i=0;i<=n-1;i++)
{
for(j=1;j<=n-1-i;j++)
{
if(a[i]>a[j])
temp=a[j];
a[j]=a[j+1];
a[j+1]=temp;
}
}
```

(Space for answers)

(1) Enter array elements : 20 31 15 9

After sorting array : 9 15 20 31

(2) There is no error.

Exercise

① The list above has 5 elements already in assorted manner so there is no comparison i.e. No of comparision is 0.

②

1000	-20	300	140	50	
Pass 1 :	-20	1000	300	140	50
	-20	300	1000	140	50
	-20	300	140	1000	50
	-20	300	140	50	1000
Pass 2 :	-20	300	140	50	1000
	-20	140	300	50	1000
	-20	140	50	300	1000
Pass 3 :	-20	140	50	300	1000
	-20	50	140	300	1000
Pass 4 :	-20	50	140	300	1000
∴ Sorted array =	-20	50	140	300	1000

XVII. Exercise

1. Find the number of comparisons required in Bubble Sort of the following list having 5 numbers.

0	1	2	3	4
10	20	30	40	50

2. Sort the given array in ascending order using Bubble Sort method and show diagrammatic representation of every iteration of for loop.

0	1	2	3	4
1000	- 20	300	140	50

3. Sort the given array in ascending order using Bubble Sort method and show diagrammatic representation of every iteration of while loop.

0	1	2	3	4
500	120	2000	-210	89

(Space for answers)

9) 500 120 2000 -210 89

Pass 1 : 500 120 2000 -210 89
89 500 2000 -210 89
120 500 -210 2000 89
120 500 -210 89 2000

Pass 2 : 120 500 -210 89 2000
120 -210 500 89 2000
-210 120 89 500 2000

Pass 3 : -210 120 89 500 2000
-210 89 120 500 2000

Pass 4 : -210 89 120 500 2000

sorted array : -210 89 120 500 2000

Practical No. 5: Program to sort an array using selection sort

I. Practical Significance:

In order to perform some operation on specific data element, the data has to be sorted in data collection, and the system requires following a Sorting method. One of the most commonly used methods is Selection Sort for sorting data from the given list.

II. Relevant Program Outcomes (POs)

- **Basic knowledge:** Apply knowledge of basic mathematics, sciences and basic engineering to solve the computer group related problems.
- **Discipline knowledge:** Apply Information Technology knowledge to solve broad-based Information Technology related problems.
- **Experiments and practice:** Plan to perform experiments and practices to use the results to solve the computer group related problems.
- **Engineering tools:** Apply relevant Computer programming / technologies and tools with an understanding of the limitations.
- **Communication:** Communicate effectively in oral and written form.

III. Competency and Practical skills

This practical expects to develop the following skills in the student.

Develop 'C' programs to solve computer group related problems.

1. Write algorithm and draw flow chart for selection sort.
2. Write / Compile / Debug and execute 'C' program for Selection sort.

IV. Relevant Course Outcome(s)

- Apply different searching and sorting techniques.

V. Practical Outcome (PrOs)

- Implement a 'C' program to sort an array using Selection Sort method.

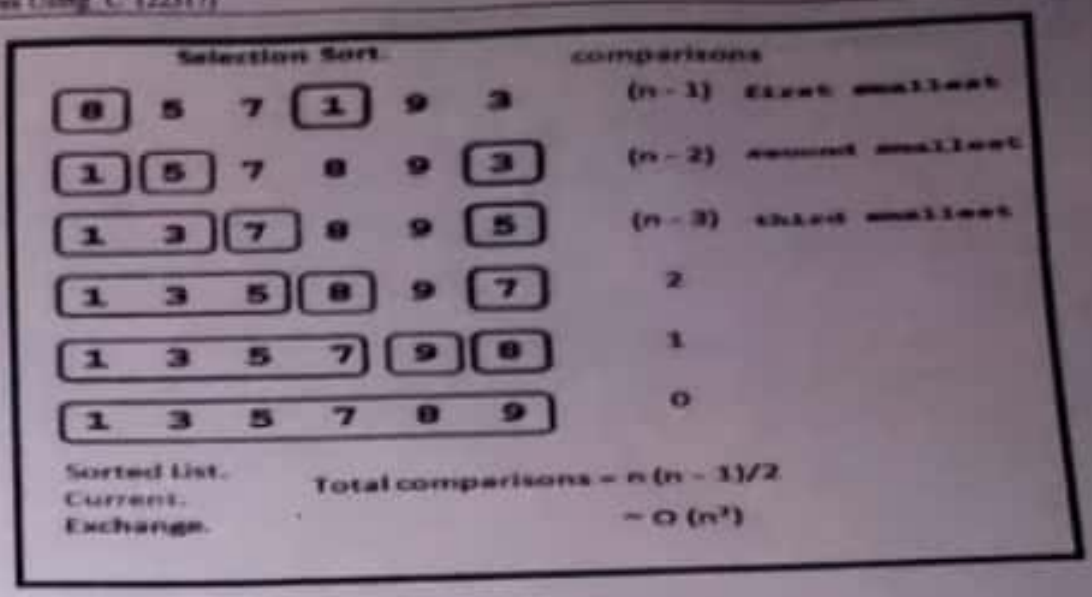
VI. Relevant Affective domain related Outcome(s)

1. Follow safety measures.
2. Follow ethical practices.

VII. Minimum Theoretical Background

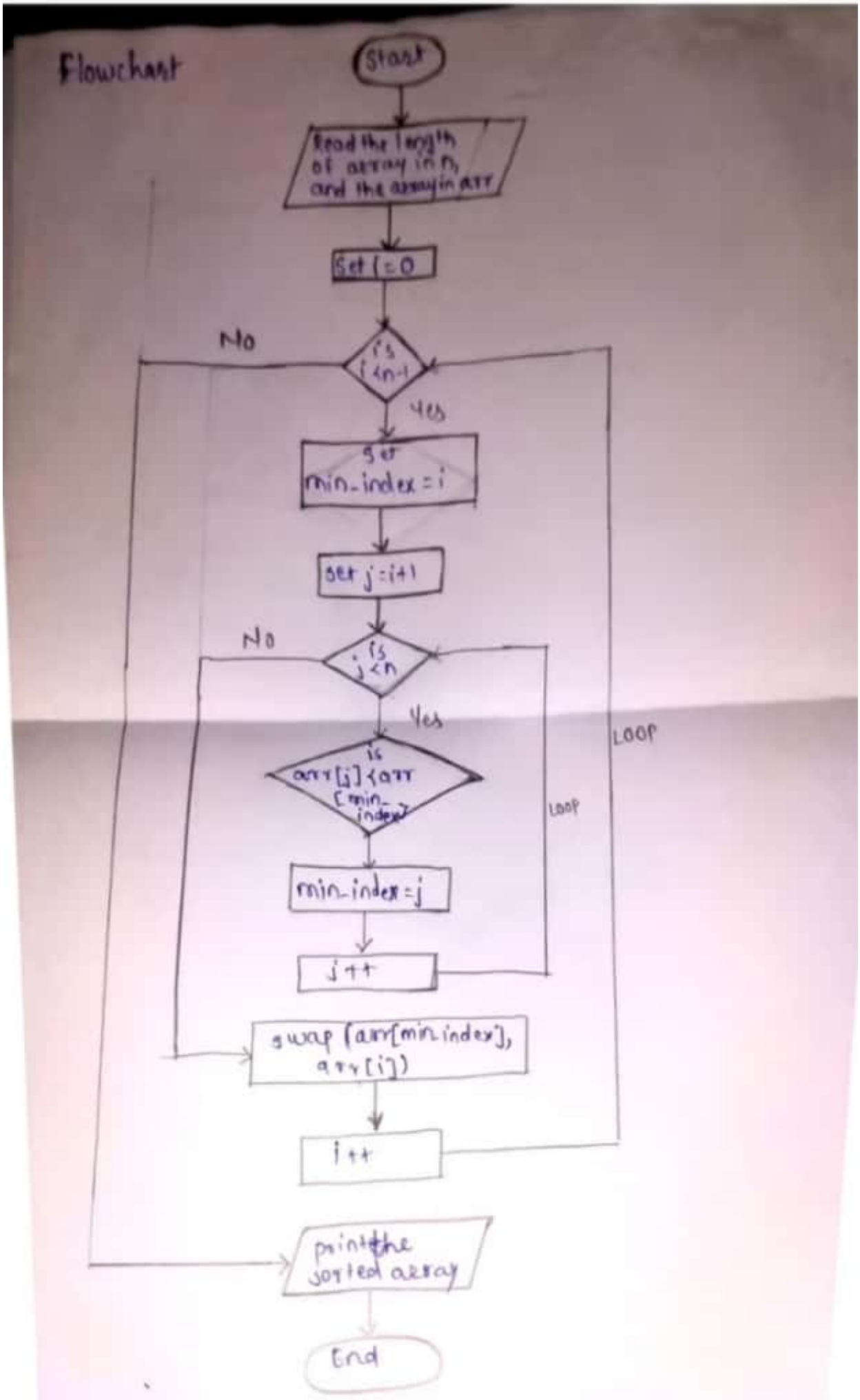
Selection sort is similar to the hand picking where we take the smallest element and put it in the first position and the second smallest at the second position and so on.
We follow the following steps to perform selection sort:

1. Start from the first element in the array and search for the smallest element in the array.
2. Swap it with the value in the first position
3. Repeat the steps above for the remainder of the list (starting at the second position and Advancing each time)



VIII. Algorithm:

1. Set min to the first location.
2. Search the minimum element in the array.
3. Swap the first location with the minimum value in the array.
4. Assign the second element as min.
5. Repeat the process until we get a sorted array.
6. Exit.



X. C Program Code:

```
#include <stdio.h>
#include<conio.h>
int main()
{
int array[100],n,c,d,position,t;
clrscr();
printf("Enter number of elements\n");
scanf("%d",&n);
printf("Enter %d integers\n",n);
for(c=0;c<n;c++)
scanf("%d",&array[c]);
for(c=0;c<(n-1);c++) // Finding minimum element (n-1) times
{
position=c;
for(d=c+1;d<n;d++)
{
if(array[position]>array[d])
position=d;
}
```

X. C Program Code:

```
#include<stdio.h>
#include<conio.h>
int main()
{
int array[100],n,c,d,position,t;
clrscr();
printf("Enter number of elements\n");
scanf("%d",&n);
printf("Enter %d integers\n",n);
for(c=0;c<n;c++)
 scanf("%d",&array[c]);
for(c=0;c<(n-1);c++) // finding minimum element (n-1) times
{
position=c;
 for(d=c+1;d<n;d++)
{
 if(array[position]>array[d])
```



```
 position=d;
}
```

```
    position [i=c)
{
 t =array[c];
array[c] = array[position];
array[position] = t;
 }
}
printf("Sorted list in ascending order:\n");
for(c=0;c<n;c++)
  printf("%d\n",array[c]);
return 0;
 getch();
}
```

2. Follow safety practices

XIII. Resources used

S. No.	Name of Resource	Specification
1	Computer Systems with broad specifications	Computer i3-i5 processor, RAM minimum 2GB and onwards.
2	Software	Turbo C/C++ version 3.0 or later version
3	Any other resource used	Windows 7 or later version/Linux version 5.0

XIV. Results (Output of the Program)

Enter number of elements : 6
Enter 6 integers : 4 2 7 1 0 3
Sorted list in ascending order : 0 1 2 3 4 7

XV. Conclusion(s)

We have perform this practical of program to sort an array using selection sort successfully.

XVI. Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so to ensure the achievement of identified CO.

Choose the right option from the following:

1 A Selection Sort compares adjacent elements, and swaps them if they are in wrong order

a. True b. False c. Depends on Elements d. None of these

Ans: a

2 For each i from 1 to n-1, there are _____ exchanges for Selection Sort

a. 1 b. n-1 c. n d. None of these

Ans: b

3 What is the output of selection sort after the 2nd iteration given the following sequence of numbers: 20 12 10 15 2

a. 2 15 12 10 20

b. 2 101 12 15 20

c. 2 12 10 15 20

d. None of the above

Ans: C

XVII. Exercise

1. Find the number of comparisons required in Selection Sort of the following given list having 5 numbers.

0	1	2	3	4
10	20	30	40	50

2. Sort the given array in ascending order using Selection Sort method and show diagrammatic representation of every iteration of for loop.

0	1	2	3	4
1900	- 20	300	140	50

3. Sort the given array in ascending order using Selection Sort method and show diagrammatic representation of every iteration of while loop.

0	1	2	3	4
500	120	2000	-210	89

(Space for answers)

① The list above has 5 elements already in sorted manner so there is no comparision i.e no of comparisions is 0.

② Pass 1 : 1000 -20 300 140 50
1000 -20 300 140 50 → -20
1000 -20 300 140 50 → -20
1000 -20 300 140 50 → -20
1000 -20 300 140 50 → 20
→ -20 1000 300 140 50

Pass 2 : -20 1000 300 140 50
-20 1000 300 140 50 → 300
-20 1000 300 140 50 → 140
-20 1000 300 140 50 → 50
→ -20 50 300 140 1000

Pass 3 : -20 50 300 140 1000
-20 50 300 140 1000 → 140
-20 50 300 140 1000 → 140
→ -20 50 140 300 1000

Pass 4 : → -20 50 140 300 1000
-20 50 140 300 1000 → 300
→ -20 50 140 300 1000

∴ Sorted array is -20 50 140 300 1000

③ Pass 1 : 500 120 2000 -210 89
500 120 2000 -210 89 → 120
500 120 2000 -210 89 → 120
500 120 2000 -210 89 → -210
500 120 2000 -210 89 → -210
→ -210 120 2000 500 89

Pass 2 : -210 120 2000 500 89
-210 120 2000 500 89 → 120
-210 120 2000 500 89 → 120
-210 120 2000 500 89 → 89
→ -210 89 2000 500 120

Pass 3 : -210 89 2000 500 120
-210 89 2000 500 120 → 500
-210 89 2000 500 120 → 120
→ -210 89 120 500 2000

Pass 4 : -210 89 120 500 2000
-210 89 120 500 2000 → 500
→ -210 89 120 500 2000

∴ Sorted array is -210 89 140 500 2000

Practical No. 6: Program to sort an array using insertion sort

I. Practical Significance:

In order to perform some operation on specific data element, the data has to be sorted in data collection, and the system requires following a Sorting method. One of the most commonly used methods is Insertion Sort for sorting data from the given list.

II. Relevant Program Outcomes (POs)

- **Basic knowledge:** Apply knowledge of basic mathematics, sciences and basic engineering to solve the computer group related problems.
- **Discipline knowledge:** Apply Information Technology knowledge to solve broad-based Information Technology related problems.
- **Experiments and practice:** Plan to perform experiments and practices to use the results to solve the computer group related problems.
- **Engineering tools:** Apply relevant Computer programming / technologies and tools with an understanding of the limitations.
- **Communication:** Communicate effectively in oral and written form.

III. Competency and Practical skills

This practical expects to develop the following skills in the student.

Develop 'C' programs to solve computer group related problems.

1. Write algorithm and draw flow chart for Insertion sort.
2. Write / Compile / Debug and execute 'C' program for Insertion sort.

IV. Relevant Course Outcome(s)

- Apply different searching and sorting techniques.

V. Practical Outcome (PrOs)

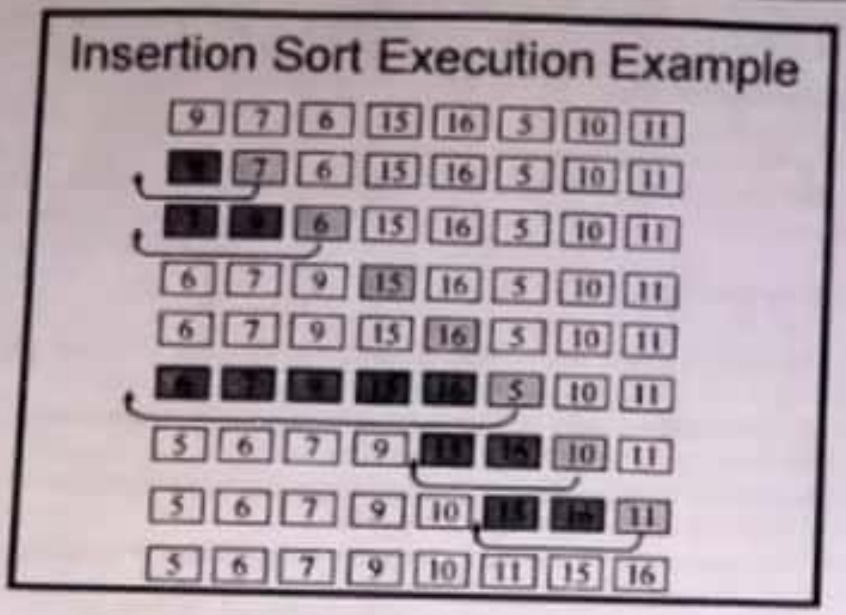
- Implement a 'C' program to sort an array using Insertion Sort method.

VI. Relevant Affective domain related Outcome(s)

1. Follow safety measures.
2. Follow ethical practices.

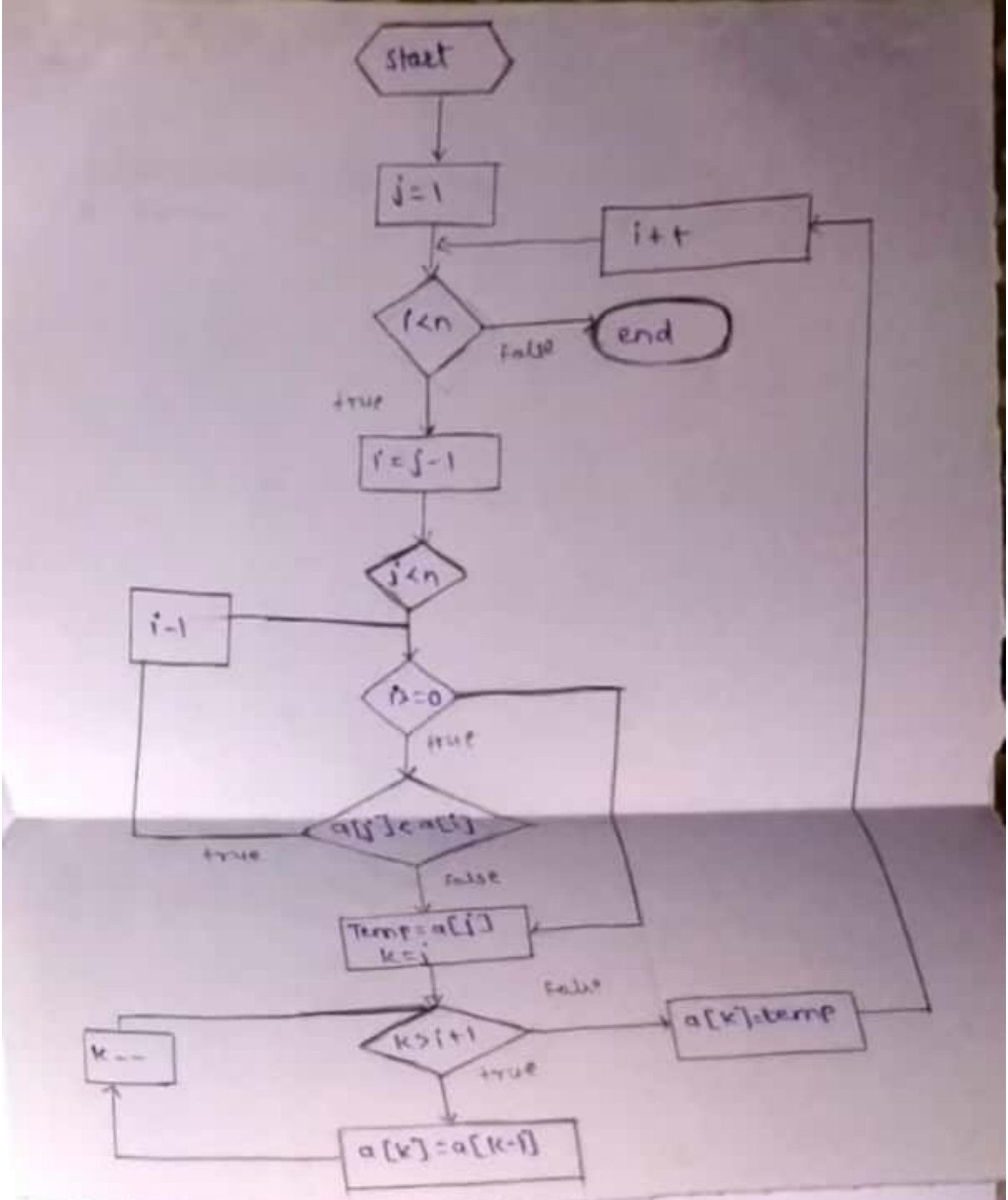
VII. Minimum Theoretical Background

Insertion Sort is a simplest data Sorting algorithm which sorts the array elements by shifting elements one by one and inserting each element into its proper position. Consider we sort playing cards in our hands.



VIII. Algorithm:

step 1 - If it is the first element, it is already sorted. return1;
step 2 - Pick next element
step 3 - Compare with all element in the sorted sub-list
step 4 - Shift all the elements in the sorted sub-list that is greater than the value to be sorted.
step 5 - Insert the value
step 6 - Repeat until list is sorted.
step 7. - Exit




```
for (i=1 ;i<=n-1; i++)
  t= array (i);
for (d=i-1 ;d>=0; d--){
```

X. C Program Code:

```
#include <stdio.h>
#include <conio.h>
int main()
{
 int n, array[1000], c, d, t, flag = 0;
 clrscr();
 printf("Enter number of elements\n");
 scanf("%d", &n);
 Print f ("Enter %d integers\n", n);
 For (c = 0; c < n; c++)
  Scanf ("%d", &array[c]);
 For (c = 1; c <= n-1; c++)
  t = array (c);
 For (d = c-1; d >= 0; d--){
  if (array[d] > t){
 array [d+1] = array [d];
  flag = 1;
 }
else
  break;
};
 if (flag)
 array [d+1] = t;
}
printf ("Sorted list in ascending order:\n");
 for (c = 0; c <= n-1; c++) {
 printf ("%d\n", array[c]);
 }
 return 0;
  getch ();
 }
```

XI. Resources required

Sr. No.	Name of Resource	Specification	Quantity	Remarks
1	Hardware: Computer System	Computer (i3-i5 preferable), RAM minimum 2 GB and onwards	As per batch size	For all Experiments
2	Operating system	Windows 7 or Later Version/LINUX version 5.0 or Later Version		
3	Software	Turbo C/C++ Version 3.0 or Later Version		

XII. Precautions

1. Save the program in specific directory / folder.
2. Follow safety practices.

XIII. Resources used

S. No.	Name of Resource	Specification
1	Computer System with broad specifications	Computer (i3-i5 preferable) RAM minimum 2 GB and onwards
2	Software	Turbo C/C++ version 3.0 or later version
3	Any other resource used	Windows 7 or later version/LINUX version 5.0 or later version

XIV. Results (Output of the Program)

Enter number of elements : 8
Enter 8 integers : 5 45 43 34 0 83 78 12
Sorted list in ascending order : 0 5 12 34 43 45 78 83

XV. Conclusion(s)

We have perform this practile of program to sort an array using insertion sort successfully.

XVI. Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

1. Define insertion sort?

Insertion sort is a simple sorting algorithm that builds the final sorted array (or list) one item at a time. It is much less efficient on large lists than more advanced algorithms such as quicksort, heapsort or mergesort.

2. Differentiate between bubble sort & insertion sort?

Bubble Sort	Insertion Sort
1. A simple sorting algorithm that repeatedly goes through the list, comparing adjacent pairs and swapping them if they are in the wrong order.	1. A simple sorting algorith that builds the final sorted list by transferring one element at a time.
2. Checks the neighbouring elements and swaps them accordingly.	2. Transfers an element at a time to the partially sorted array.
3. More number of swaps.	3. Less number of swaps
4. Simple	4. Complex than bubble sort
5. Bubble sort is slower than insertion sort.	5. Insertion sort is twice as fast as bubble sort.

Choose the right option from the following:-

3. Insertion sort is 1.in-place 2.out-place 3.stable 4.non-stable

a.1 and 3 b.1 and 4 c.2 and 3 d.2 and 4

Ans: a.

4. Sorting of playing cards is an example of

a. Bubble sort b. Insertion sort c. Selection sort d. Quick sort

Ans: a.

Insertion sort is a simple sorting algorithm that builds the final sorted array (or list) one item at a time. It is much less efficient on large lists than more advanced algorithms such as quicksort, heapsort or mergesort.

2. Differentiate between bubble sort & insertion sort?

Bubble Sort	Insertion Sort
1. A simple sorting algorithm that repeatedly goes through the list, comparing adjacent pairs and swapping them if they are in the wrong order.	1. A simple sorting algorithm that builds the final sorted list by transferring one element at a time.
2. Checks the neighbouring elements and swaps them accordingly.	2. Transfers an element at a time to the partially sorted array.
3. More number of swaps.	3. Less number of swaps
4. Simple	4. Complex than bubble sort
5. Bubble sort is slower than insertion sort.	5. Insertion sort is twice as fast as bubble sort.

Choose the right option from the following:-

3. Insertion sort is 1.in-place 2.out-place 3.stable 4.non-stable

a.1 and 3 b.1 and 4 c.2 and 3 d.2 and 4

Ans: a.

4. Sorting of playing cards is an example of

a. Bubble sort b. Insertion sort c. Selection sort d. Quick sort

Ans: a.

Practical No. 7: Perform push and pop operations on stack

I. Practical Significance:

In order to perform insertion and deletion of data items from given data list based on Last in First out (LIFO) suitable data structure is required. So Stack is a data structure that follows the LIFO order and performs these operations from one end.

II. Relevant Program Outcomes (POs)

- **Basic knowledge:** Apply knowledge of basic mathematics, sciences and basic engineering to solve the computer group related problems.
- **Discipline knowledge:** Apply Information Technology knowledge to solve broad-based Information Technology related problems.
- **Experiments and practice:** Plan to perform experiments and practices to use the results to solve the computer group related problems.
- **Engineering tools:** Apply relevant Computer programming / technologies and tools with an understanding of the limitations.
- **Communication:** Communicate effectively in oral and written form.

III. Competency and Practical skills

This practical expects to develop the following skills in the student.

Develop 'C' programs to solve computer group related problems.

1. Write algorithm and draw flow chart of stack for push and pop operations.
2. Compile/Debug/Save the 'C' program.

IV. Relevant Course Outcome(s)

- Implement basic operations on stack and queue using array representation.

V. Practical Outcome (POs)

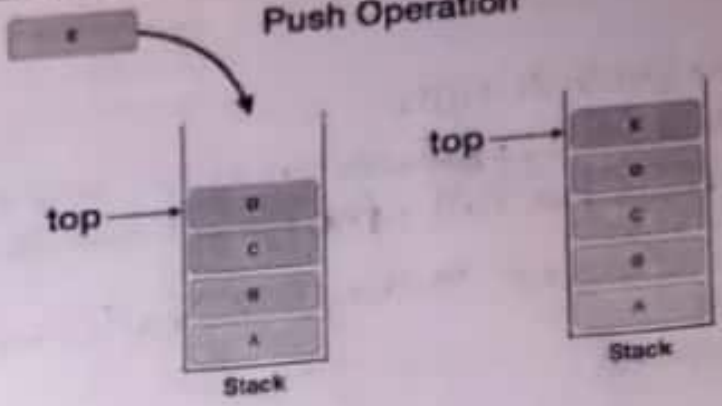
- Write 'C' program to perform PUSH and POP operations on stack array.

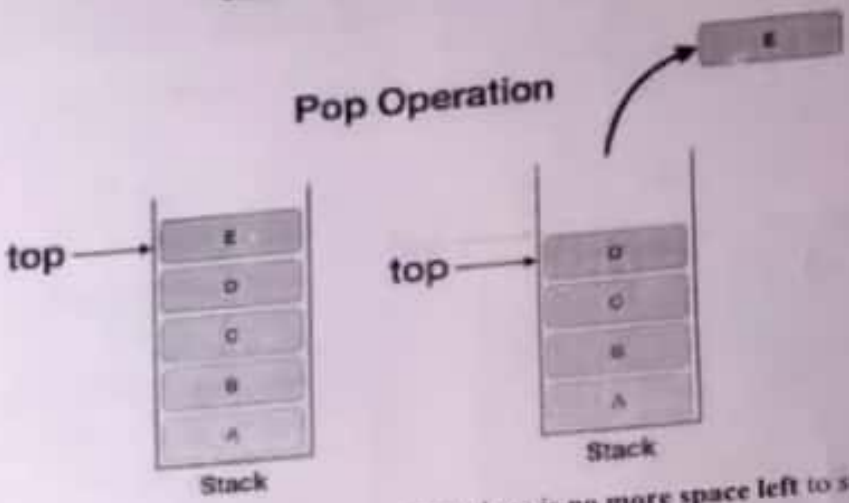
VI. Relevant Affective domain related Outcome(s)

1. Follow ethical practices.

VII. Minimum Theoretical Background

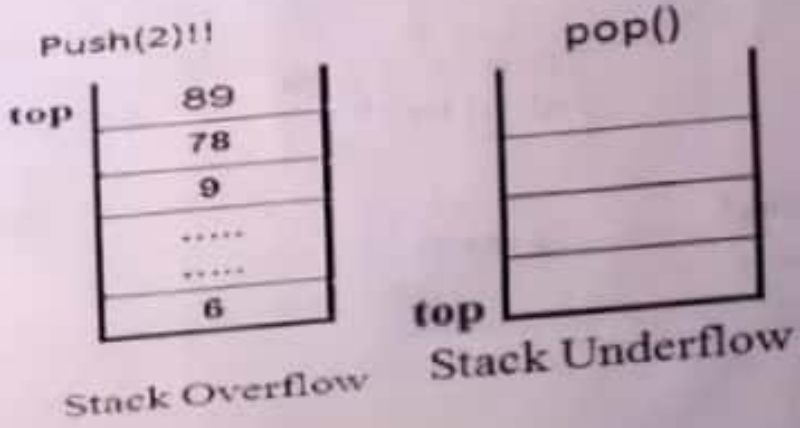
A stack is a container of data items that are inserted and removed according to the last-in first-out (LIFO) principle. A stack has a restriction that insertion and deletion of element can only be done from only one end of stack and we call that position as **top**. The element at top position is called **top of the stack**. Insertion of element is called **PUSH** and deletion is called **POP**.





Overflow Condition: Happens when there is **no more space left** to store a **data item** that is **pushed.**

Underflow Condition: Happens when the **stack** is **empty** and the user executed a **POP** operation.



VIII. Algorithm

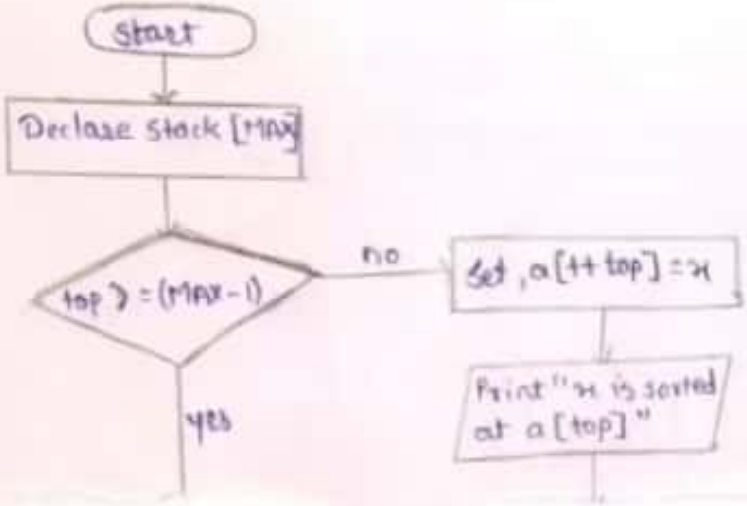
For Push Operation -

Step 1 - Checks if the stack is full.
Step 2 - If the stack is full, produces an error and exit.
Step 3 - If the stack is not full, increments top to point next empty space.
Step 4 - Adds data element to the stack location, where top is pointing.
Step 5 - Returns success.

For Pop Operation -

Step 1 - Checks if the stack is empty.
Step 2 - If the stack is empty, produces an error and exit.
Step 3 - If the stack is not empty, accesses the data element at which top is pointing.
Step 4 - Decreases the value of top by 1.
Step 5 - Returns success.

PUSH()



X. C Program Code

```
#include<stdio.h>
#include<conio.h>
#define size 10
struct stack
{
  int item[size];
  int top;
};
void init(struct stack*);
void push(struct stack*,int);
int pop(struct stack*);
int isempty(struct stack*);
int isfull(struct stack*);
int print(struct stack*);
void main()
{ struct stack s;
  int val,choice;
  init(&s);
  do
  {printf("\n1 -> push");
  printf("\n2 -> pop");
  printf("\n3 -> print");
  printf("\n 4 -> exit");
  printf("\n Enter your choice :");
  scanf("%d",&choice);
  switch(choice)
  {
  case 1:printf("\n Enter value tobe push:");
  scanf("%d",&s,val);
  push(&s,val);
  break;
  case 2: val=pop(&s);
  printf("\n poped value is %d",
  val);
  break;
  case 3: val=print(&s);
  printf("pop value = %d",val);
  break;
case 4: break;
} while(choice!=4);
}
void init(struct stack*s)
{s->top=-1;}
int isempty(struct stack*s)
{if(s->top==-1)
   return(1);
  else
   return(0);}
int isfull(struct stack*s)
{
 if(s->top==size-1)
  return(1);
 else
   return(0);}
int pop(struct stack*s)
{int val;
if(isempty(s))
{printf("stack is empty");
exit(0);
}
val=s->info[s->top];
s->top=s->top-1;
return(val);
}
```

XI. Resources required

```
int print(struct stack*s)
{ int val;
 if(isempty(s))
 {
  printf("stack is empty");
  exit(0);
 }
 val=s->info[s->top];
 return(val);
}
void push(struct stack*s,int val)
{ if(isfull(s))
 {
 printf("\n stack is full");
 exit(0);
 }
 (s->top)++;
 s->item[s->top]=val;
```

returns success.

For POP Operation -

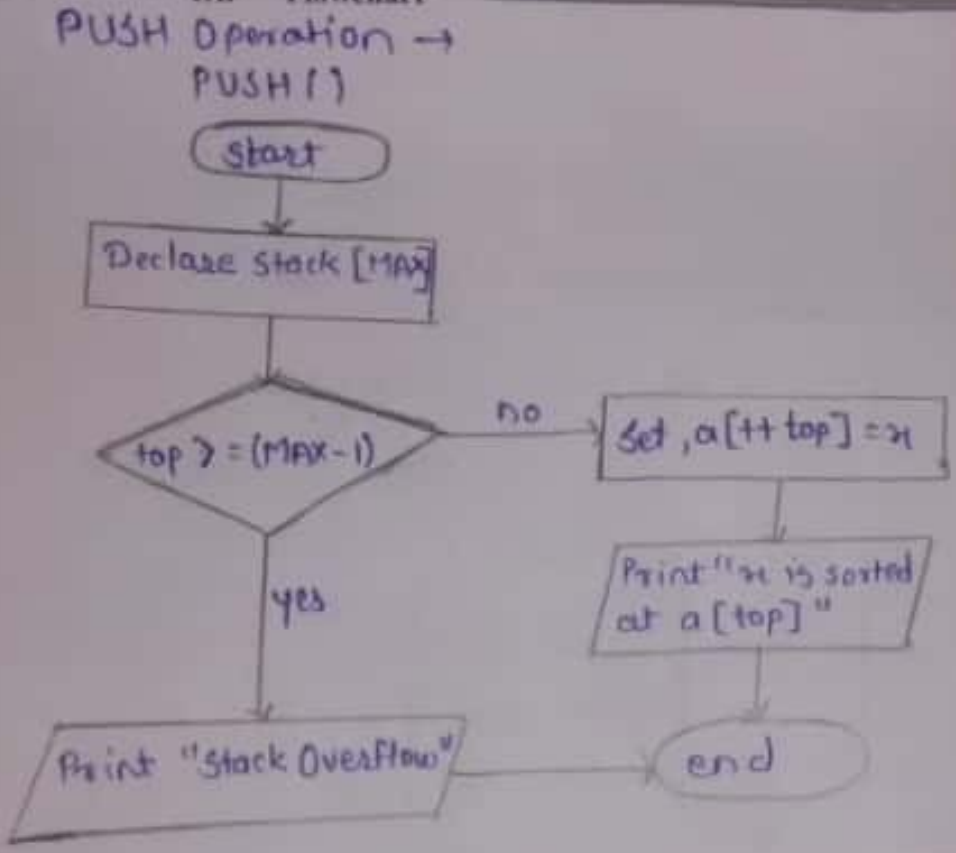
Step 1 - Checks if the stack is empty.

Step 2 - If the stack is empty, produces an error and exit.

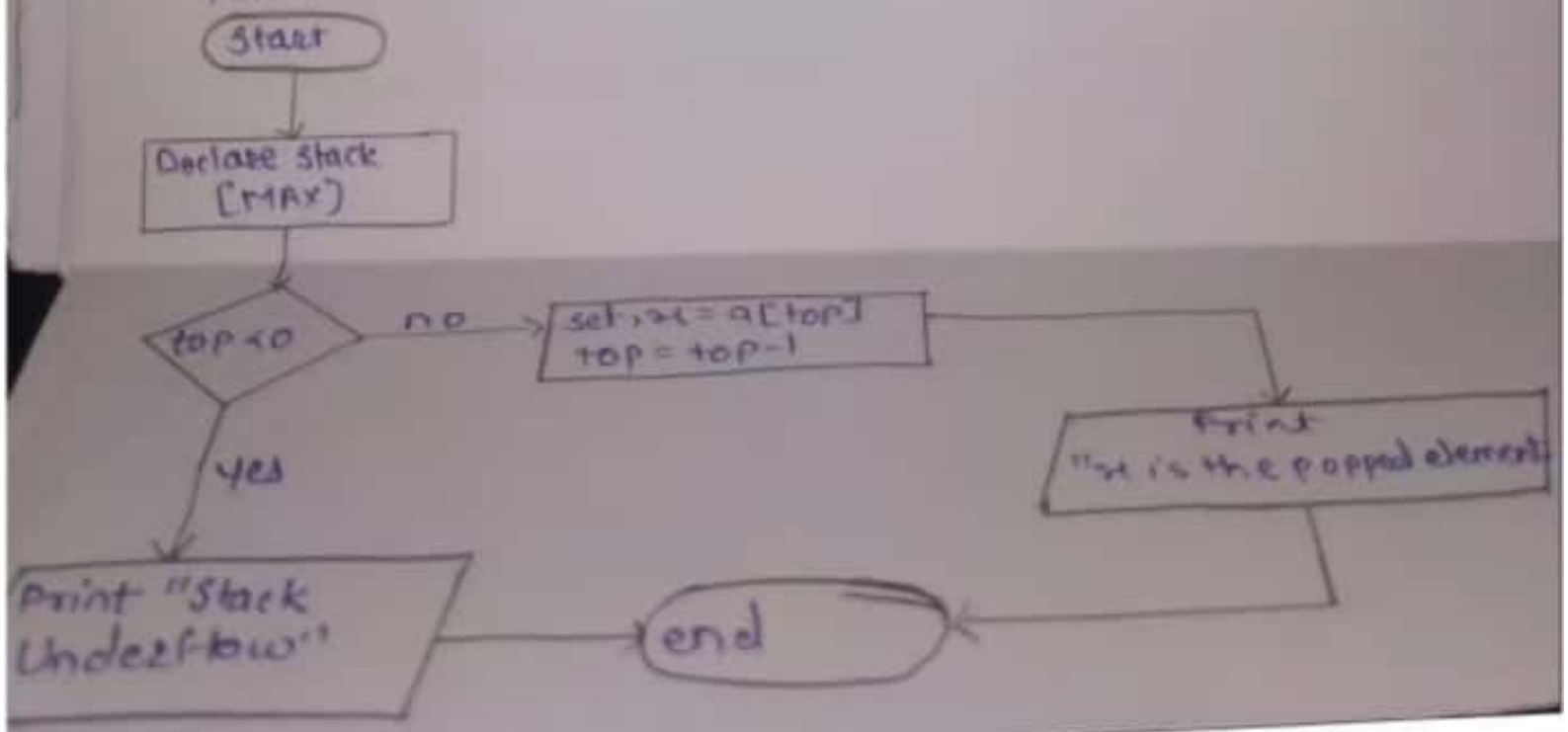
Step 3 - If the stack is not empty, accesses the data element at which top is pointing

i.e. Flowchart

PUSH Operation →



POP() Operation:



XII. Precautions

1. Give proper max-size for stack
2. Programs should be saved in desired folders.

XIII. Resources used

S. No.	Name of Resource	Specification
1	Computer System with broad specifications	Computer (i3-i5 preferable) RAM minimum 2GB and onwards.
2	Software	Turbo C/C++ version or 5.0 version, or later ; gcc compiler.
3	Any other resource used	Windows 7 or later version.

XIV. Result (Output of the Program)

.....

.....

.....

.....

XV. Conclusion(s)

We have performed this practical to perform push and pop operations on stack successfully.

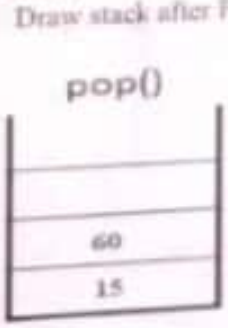
.....

XVI. Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: Programming exercise use blank pages provided or attach more pages if needed.)

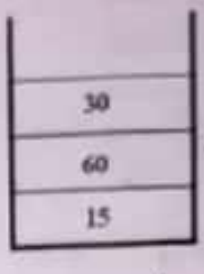
1) Draw stack after POP operation using given below diagram.



Ans:

2) Draw stack after PUSH operation using given stack

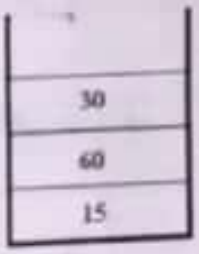
PUSH(45)



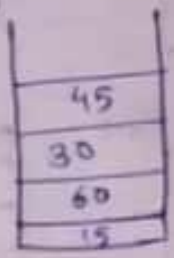
Ans

3) Write a TOP value after PUSH operation.

PUSH(45)



Ans



XVII. Exercise

(Note: Programming exercise use blank pages provided or attach more pages if needed.)

1) Perform following operation on stack of size 5.
   PUSH(30)
   PUSH(12)
   PUSH(10)
   POP()
   PUSH(45)
   POP(70)

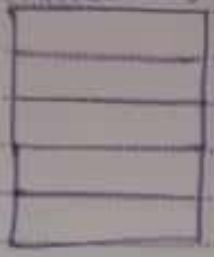
2) Implement a "C" program to stack size-8 for push(10), push(20), pop, push(10), push(20), pop, pop, pop, push(20), pop and draw final output.

3) Perform following operation on stack of size 5.
   PUSH(100)
   PUSH(120)
   PUSH(50)
   POP()
   POP()
   POP(70)

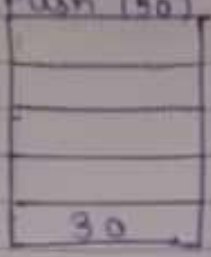
(Space for answers)

(7)

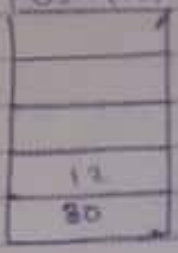
initialize stack



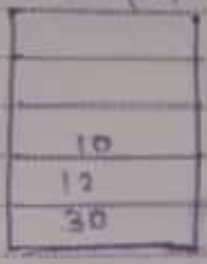
Push (30)



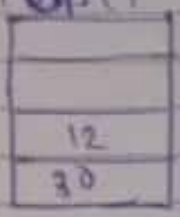
PUSH (12)



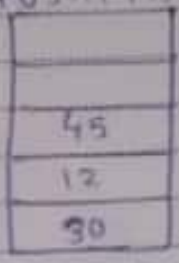
PUSH(10)



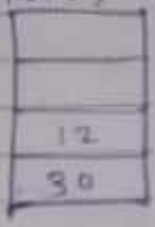
POP()



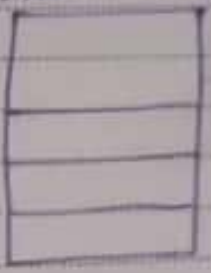
PUSH(45)



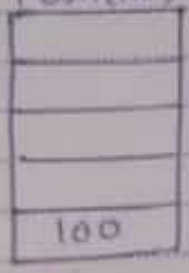
POP()



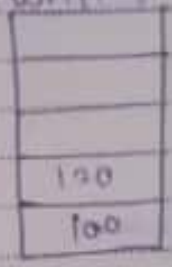
(9) Initialize stark



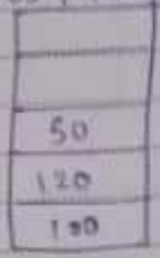
PUSH(100)



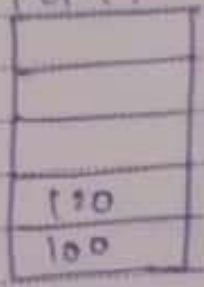
PUSH(120)



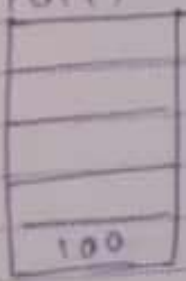
PUSH (50)



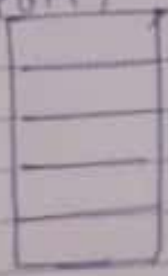
POP()



POP()



POP()



∴ at last stack is empty

Practical No. 8: Perform insert and delete operations on linear queue using array

I. Practical Significance:

In order to perform INSERT and DELETE operations from two ends on given list based on First in First out (FIFO) order. Linear Queue is collection of elements that follows the FIFO order which keep track of two indices, front and rear one for Insert and another for Delete.

II. Relevant Program Outcomes (POs)

- **Basic knowledge:** Apply knowledge of basic mathematics, sciences and basic engineering to solve the computer group related problems.
- **Discipline knowledge:** Apply Information Technology knowledge to solve broad-based Information Technology related problems.
- **Experiments and practice:** Plan to perform experiments and practices to use the results to solve the computer group related problems.
- **Engineering tools:** Apply relevant Computer programming / technologies and tools with an understanding of the limitations.
- **Communication:** Communicate effectively in oral and written form.

III. Competency and Practical skills

This practical expects to develop the following skills in the student.

Develop 'C' programs to solve computer group related problems.

1. Write algorithm and draw flow chart of linear queue for Insert and Delete operations.
2. Compile/Debug/ save the 'C' program.

IV. Relevant Course Outcome(s)

- Implement basic operations on stack and queue using array representation.

V. Practical Outcome (PrOs)

- Write 'C' program to perform INSERT and DELETE operations on linear queue using Array Part-I and II.

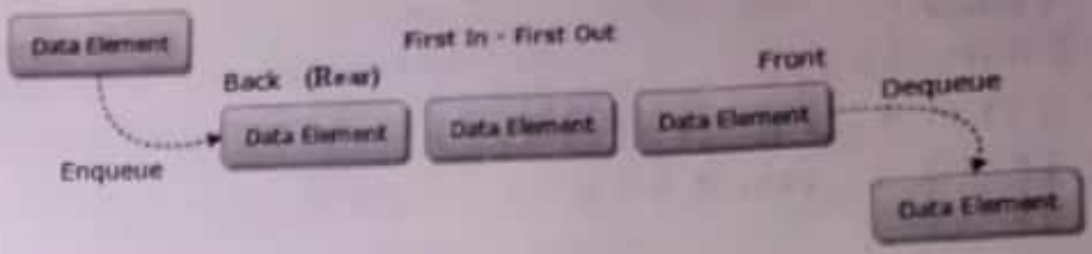
VI. Relevant Affective domain related Outcome(s)

1. Follow ethical practices.

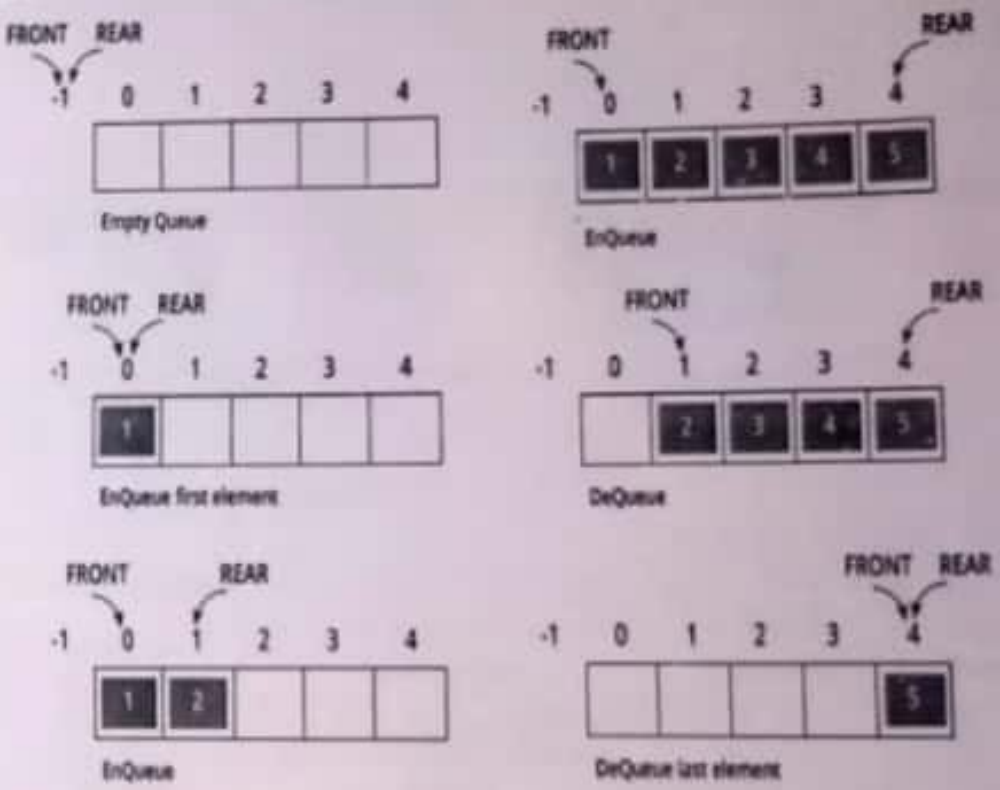
VII. Minimum Theoretical Background

A queue is a linear list of elements in which deletion of an element can take place only at one end called the front and insertion can take place on the other end which is termed as rear. The term front and rear are used frequently while describing queues in a linked list. In a queue the first item inserted is the first to be removed (First-In-First-

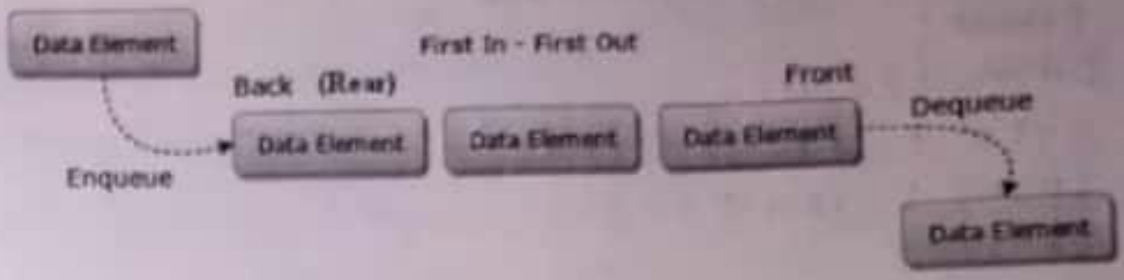
Out, FIFO). A queue has two pointers front and rear, pointing to the front and rear elements of the queue, respectively. This is a linear list DATA STRUCTURE used to represent a linear list and permits deletion to be performed at one end of the list and the insertion at the other end.



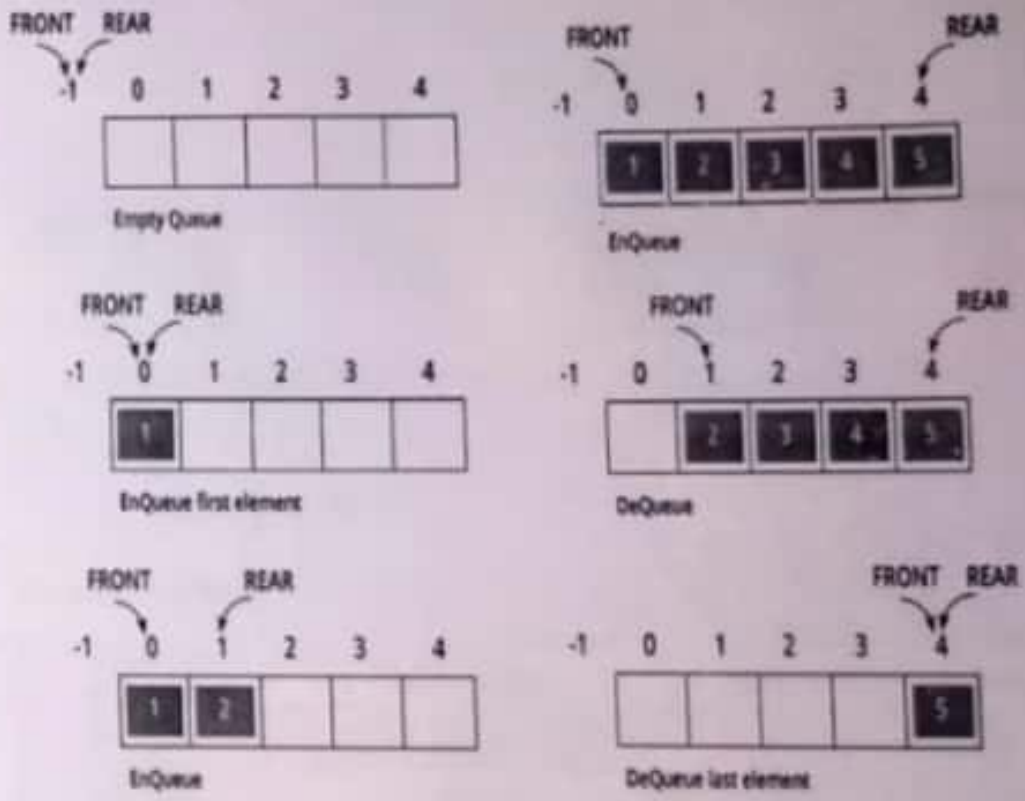
STEP BY STEP PROCESS IN LINEAR QUEUE:



Out, FIFO). A queue has two pointers front and rear, pointing to the front and rear elements of the queue, respectively. This is a linear list DATA STRUCTURE used to represent a linear list and permits deletion to be performed at one end of the list and the insertion at the other end.



STEP BY STEP PROCESS IN LINEAR QUEUE:



VIII. Algorithm

Delete Operation -

IF (FRONT = REAR + 1) then
Step 1 - Display "Queue underflow";
Step 2 - Return;
Otherwise / else
Step 3 - ITEM := QUEUE (FRONT);
Step 4 - FRONT := FRONT + 1;
Step 5 - Return;

Insert Operation -

Step 1 - IF (REAR = MAX) then
Step 2 - Display "Queue Overflow";
Step 3 - Return;
Step 4 - else
Step 5 - REAR := REAR + 1;
Step 6 - QUEUE (REAR) := ITEM;
Step 7 - Return;

X. C Program Code

```
# include<stdio.h>
# include<conio.h>
# define size 20
struct queue
{
  int item[size];
  int rear, front;
};
void init(struct queue *q)
{
  q->rear = -1;
  q->front = -1;
};
int isempty(struct queue *q)
{
  if (q->rear == q->front)
    return(1);
  else return(0);
}
int isfull(struct queue *q)
{
  if (q->rear == size-1)
    return(1);
  else return(0);
}
void enqueue(struct queue *q, int val)
{
  if (isfull(q))
  {
    printf("\n queue is full");
    exit(0); }
  q->rear = q->rear+1;
  q->item[q->rear] = val;
}
int dequeue(struct queue *q)
{
  int val;
  int (isempty)(q))
  {
    printf("\n queue is empty");
    exit(0);
  }
  q->front = q->front+1;
  val = q->item[q->front];
  return(val);
}
int peek(struct queue *q)
{
  int val;
  if (isempty(q) || q->front == -1)
  {
    printf("\n queue is empty");
    exit(0);
  }
  val = q->item[q->front];
  return(val);
}
void main()
{
  struct queue q;
  int val, choice;
  init(&q);
  do
  {
    printf("\n1→Insert value into queu
    printf("\n2→Delete value from qu
    printf("\n3→Peek value from q
```

XI. Resources required

Sr. No.	Name of Resource	Specification	Quantity	Remarks
1	Hardware: Computer System	Computer (i3-i5 preferable), RAM minimum 2 GB and onwards	As per batch size	For all Experiments
2	Operating system	Windows 7 or Later Version/LINUX version 5.0 or Later Version		
3	Software	Turbo C /C++ Version 3.0, or later , gcc compiler		

XII. Precautions

1. Programs should be save in desired location.

XIII. Resources used

S. No.	Name of Resource	Specification
1	Computer System with broad specifications	Computer (i3-i5 Preferable), RAM minimum 2 GB and ownwards.
2	Software	Turbo C/C++ version 3.0, BT later, gcc compiler.
3	Any other resource used	Windows 7 or later version

XIV. Result (Output of the Program)

.....

.....

.....

.....

XV. Conclusion(s)

We have performed this practicle to perform insert and delete operations on linear queue using array successfully.

.....

XVI. Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

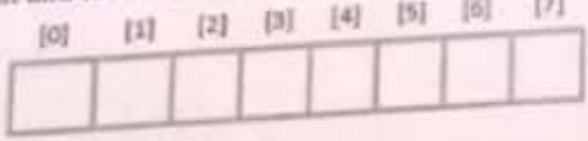
(Note: Programming exercise use blank pages provided or attach more pages if needed.)

1) Write a front and rear values of linear queue.

Enqueue Dequeue

Ans:-

2) Sketch the front and rear in empty queue.



Ans:-

3) Sketch the front and rear in this queue.

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
27	19	17	7				

Ans:-

(Space for Answer)

(1)

Enqueue Deque

5	6	5	8	9	10	6	7

Front = -1 , rear = 7 , value = 7

(2) rear = -1 , Front = -1

(3) a) Rear = 0 Front = -1 value = 27

[0]	[1]	[2]	[3]	[4]	[4]	[6]	[7]
27							

b) Rear 1 Front = -1 value = 19

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
29	19						

c) Rear = 2 Front = -1 value = 17

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
29	19	17					

d) Rear = 3 , Front = -1 value = 7

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
27	19	17	7				

```
printf ("\n Enter your choice");
scanf ("%d", &choice);
switch (choice)
{
case1:
printf ("\n Enter value:");
scanf ("%d", &val);
enqueue (&q, val);
break;
case2:
val = dequeue (&q);
printf ("\n Deleted value = %d" val);
break;
case3:
val = peek (&q);
printf ("\n Peeked value = %d", val);
break;
case 4: break;
}while (choice! = 4);
}
```

XV. Conclusion(s)

We have performed this practicle to perform insert and delete operations on linear queue using array successfully.

XVI. **Practical Related Questions**

Note: Below given are few sample questions for reference. Teacher must desig more such questions so as to ensure the achievement of identified CO.

(Note: Programming exercise use blank pages provided or attach more pages needed.)

front and rear values of linear queue.

Practical No. 9: Perform insert and delete operations on circular queue using array

Practical Significance:

In order to perform INSERT and DELETE operations from two ends on given list based on First in First out (FIFO) order. But in Linear queue deleted item's location is wasted because another data item cannot occupy that location. So to overcome this Circular Queue can be the feasible option.

Relevant Program Outcomes (POs)

- **Basic knowledge:** Apply knowledge of basic mathematics, sciences and basic engineering to solve the computer group related problems.
- **Discipline knowledge:** Apply Information Technology knowledge to solve broad-based Information Technology related problems.
- **Experiments and practice:** Plan to perform experiments and practices to use the results to solve the computer group related problems.
- **Engineering tools:** Apply relevant Computer programming / technologies and tools with an understanding of the limitations.
- **Communication:** Communicate effectively in oral and written form.

Competency and Practical skills

This practical expects to develop the following skills in the student.

Develop 'C' programs to solve computer group related problems.

1. Write algorithm and draw flow chart of circular queue for Insert and Delete operations.
2. Compile/Debug/ Save the 'C' program.

Relevant Course Outcome(s)

- Implement basic operations on stack and queue using array representation.

Practical Outcome (PrOs)

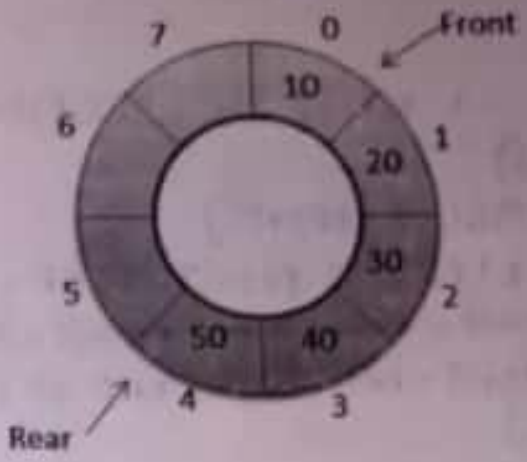
- Write 'C' program to perform INSERT and DELETE operations on circular queue using Array Part-I and II.

Relevant Affective domain related Outcome(s)

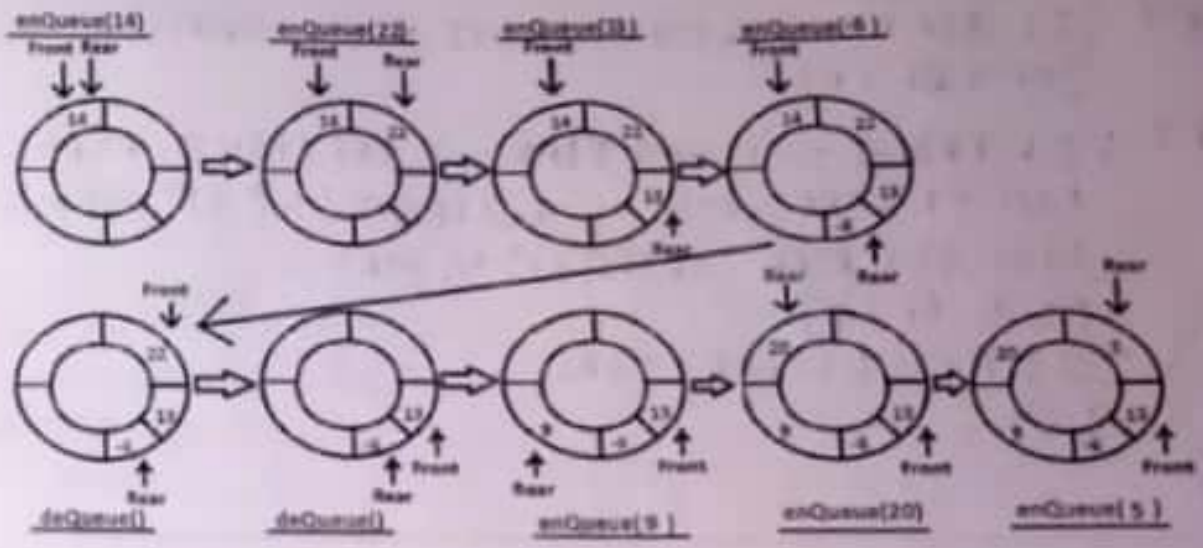
1. Follow ethical practices.
2. Follow Safety practices

Minimum Theoretical Background

Circular queue is a linear data structure. It follows FIFO principle. In circular queue the last node is connected back to the first node to make a circle. Elements are added at the rear end and the elements are deleted at front end of the queue. Any position before front is also after rear. It is also called '**Ring Buffer**'.



STEP BY STEP PROCESS:



- **Front:** Get the front item from queue.
- **Rear:** Get the last item from queue.
- **enQueue (value)** This function is used to insert an element into the circular queue. In a circular queue, the new element is always inserted at Rear position.
- **deQueue()** This function is used to delete an element from the circular queue. In a circular queue, the element is always deleted from front position.

VIII. Algorithm

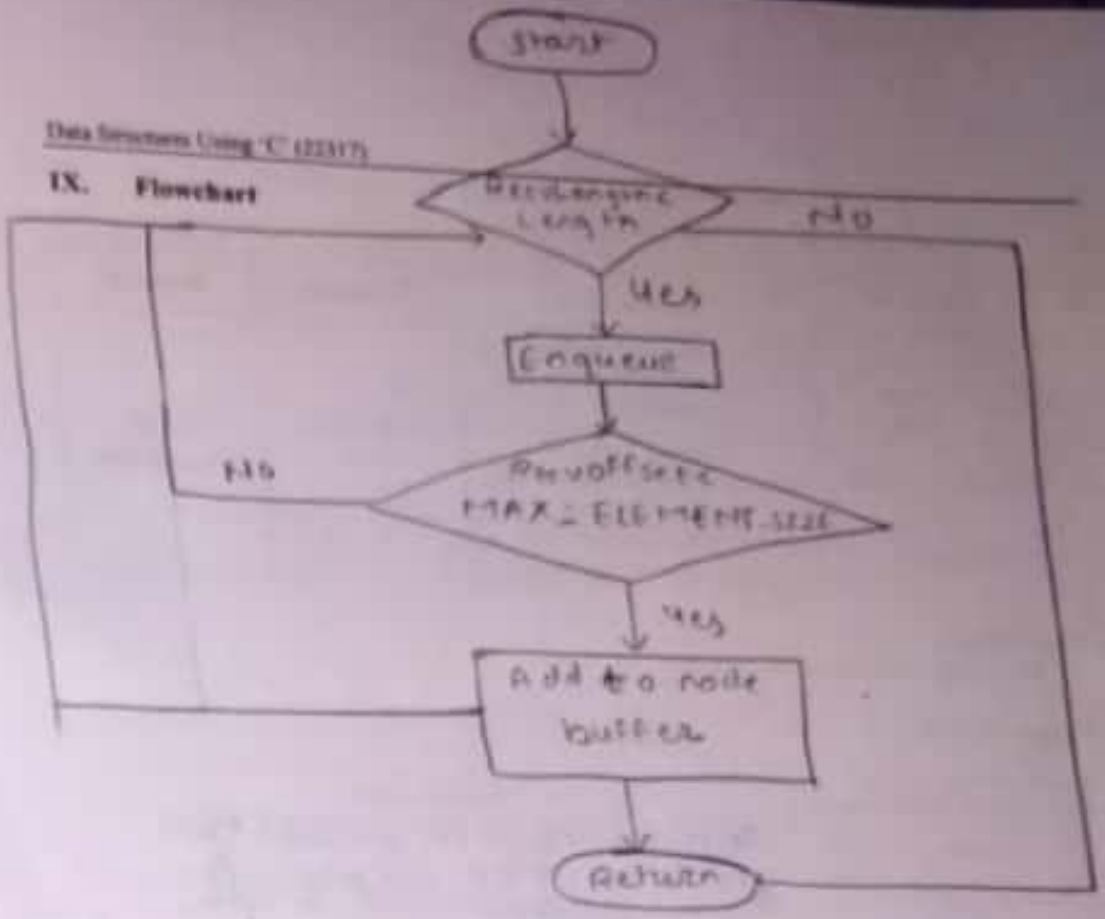
Delete Operation

Step 1 : IF FRONT = -1 Write "UNDERFLOW" Goto step 4
[END OF IF]
Step 2 : SET VAL = QUEUE [FRONT]
Step 3 : IF FRONT = REAR SET FRONT = REAR = -1
ELSE IF FRONT = MAX-1 SET FRONT = 0
ELSE SET FRONT = FRONT + 1 [END OF IF]
[END OF IF]
step 4 : Exit

Insert Operation

Step 1 : IF (REAR+1) % MAX = FRONT Write "OVERFLOW" Goto step 4
[end of IF]
Step 2 : IF FRONT = -1 and REAR = -1 SET FRONT = REAR = 0
ELSE IF REAR = MAX-1 and FRONT != 0 SET REAR = 0
ELSE SET REAR = (REAR+1) % MAX
[END OF IF]
Step 3 : SET QUEUE [REAR] = VAL
step 4 : Exit

IX. Flowchart



X. C Program Code

```
#include <stdio.h>
#include <conio.h>
#define size 20
struct queue
{
int item[size];
int rear, front;
};
void init (struct queue *);
void insert (struct queue *, int);
int delete (struct queue*);
int isempty (struct queue*);
int isfull (struct queue *);
int peek (struct queue*);
void main(){
int val, choice;
struct queue q;
init (&q)
do{
printf ("\n1 -> insert");
printf ("\n2 -> delete");
printf ("\n3 -> peek");
printf ("\n4 -> exit");
printf ("\n enter your choice:");
scanf ("%d", &choice);
switch (choice)
{
case 1:
printf ("\n enter value :");
scanf ("%d", &val);
insert (&q, val);
break;
case 2:
val = delete (&q);
printf ("\n deleted value is %d", val);
break;
case 3:
val = peek (&q);
printf ("\n peeked value is %d", val);
break;
case 4: break;
}
getch();
}while(choice!=4)
}
void init (struct queue *q)
{
q->rear = size-1;
q->front = size-1;
}
void insert(struct queue * q, int
{
```

XI. Resources required

Sr. No.	Name of Resource	Specification	Quantity	Remarks
1	Hardware: Computer System	Computer (i3-i5 preferable), RAM minimum 2 GB and onwards	As per batch size	For all Experiments
2	Operating system	Windows 7 or later Version/LINUX version 5.0 or later Version		
3	Software	Turbo C /C++ Version 3.0, or later , gcc compiler		

XII. Precautions

1. Save program in dedicated folder.

XIII. Resources used

S. No.	Name of Resource	Specification
1	Computer System with broad specifications	Computer (i3-i5 preferable) RAM minimum 2 GB and onwards.
2	Software	Turbo C/C++ Version 3.0 ,or later, gcc compiler
3	Any other resource used	Windows 7 or later version/LINUX version 5.0 or later version.

XIV. Result (Output of the Program)

XV. Conclusion(s)

We have performed this practice to perform insert and delete operations on circular queue using array successfully.

XVI. Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: Programming exercise use blank pages provided or attach more pages if needed.)

1) Write a front and rear values of circular queue.

Ans:

2) Show the front and rear in empty circular queue

Ans:

3) Show the circular queue using given values of array (size is 5).
15, 20, 30,10,5

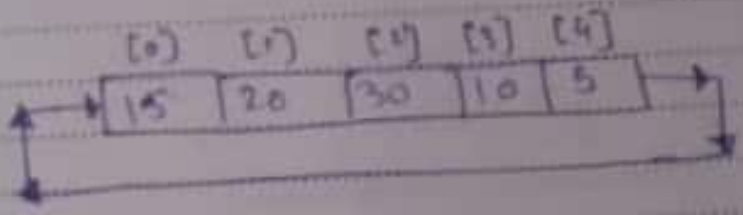
Ans:-

(Space for Answer)

(1) Front = [4] → 50, rear = [0] → 10

(2) Front = rear = -1

(3)



```
if ((q->rear +1) %.size == q->front)
{
printf ("\n queue is full");
exit (0);
}
q->rear = (q->rear +1) %. size;
q->item [q->rear] = val;
}
int delete (struct queue *q)
{
int val;
if (q->rear == q->front)
{
printf ("\n queue is empty! ");
exit (0);
}
q->front = (q->front +1) %. size;
val = q-> item (q->Front);
return(val);
}
int isempty (struct queue *q)
{
if (q->rear == q->front)
return (1);
else
return(0);
}
int isfull (struct queue *q)
{
if (q->rear == size -1)
return(1);
else
return(0);
}
```

Practical No. 10: Perform operation on singly linked list

I. Practical Significance

In order to use and perform operations on dynamic memory allocation in application non-contiguous data structure is required. So linked list is a used to Organize data in non-contiguous form. This will save wastage of memory by using dynamic memory allocation.

II. Relevant Program Outcomes (POs)

- **Basic knowledge:** Apply knowledge of basic mathematics, sciences and basic engineering to solve the broad-based Computer engineering problem.
- **Discipline knowledge:** Apply Information Technology knowledge to solve broad- based Information Technology related problems.
- **Experiments and practice:** Plan to perform experiments, practices and to use the results to solve Information Technology related problems.
- **Engineering tools:** Apply appropriate Computer Engineering / Information Technology related techniques/ tools with an understanding of the limitations.
- **Communication:** Communicate effectively in oral and written form.

III. Competency and Practical skills

This practical is expect to develop the following skills in student.

Develop 'C' programs to solve broad-based computer group related problems.

1. Write algorithm and draw Flow Chart for inserting and deleting nodes in singly linked list.
2. Write C program for inserting and deleting nodes in singly linked list.
3. Compile/Debug/Save the C program.

IV. Relevant Course Outcome(s)

- Implement basic operations on Linked List.

V. Practical Outcome

- Write C program to perform the operations(Insert, Delete, Traverse and Search) on singly linked list.

VI. Relevant Affective domain related Outcome(s)

1. Follow ethical practices.
2. Follow safety practices

VII. Minimum Theoretical Background

Linked List: Linked List data structure consists of collection of nodes in a sequence which is divided in two parts. Each node consists of its own data and the address of the next node and forms a chain. Linked Lists are used to create stacks, queues,trees and graphs.

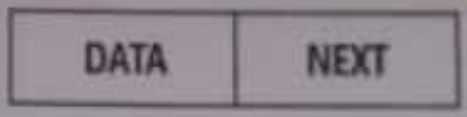


Fig1. Node

Data holds the data variable while Next holds the address to the next Node in the list.

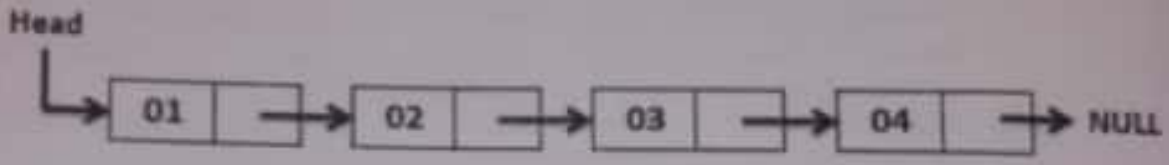


Fig 2. Linked List

Head is a pointer variable of type struct Node which acts as the Head(starting of a node) to the list. Initially we set 'Head' as NULL which means list is empty. Basically Single Linked Lists are uni-directional as they can only point to the next Node in the list but not to the previous. The operations we can perform on singly linked lists are insertion, deletion and traversal.

VIII. Algorithm (Attach separate paper if required)

1. **Insert a node**
   - i. At Beginning of Linked List
   - ii. At the end of Linked List
   - iii. At a given position in Linked List

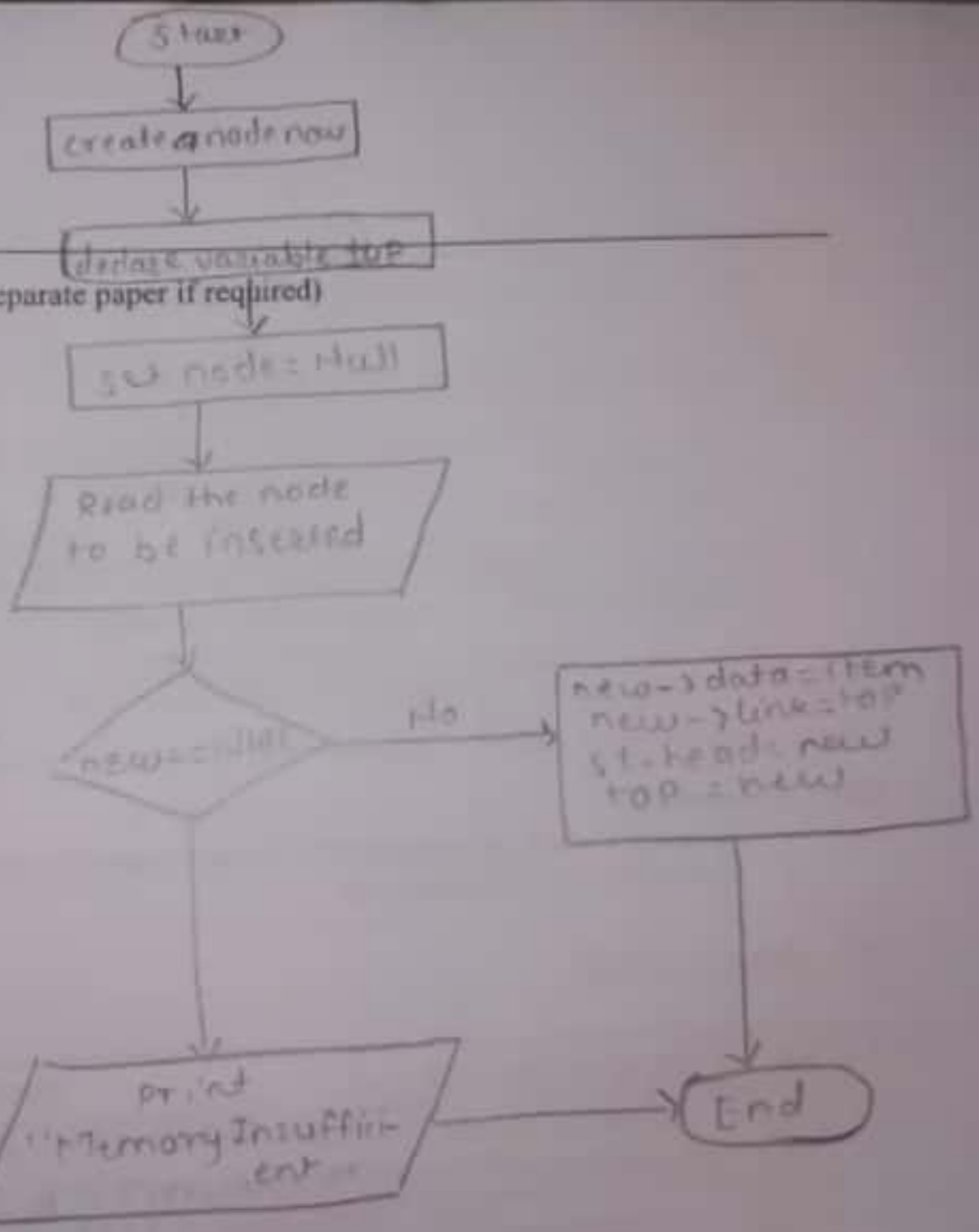
2. **Delete a node**
   - i. At Beginning of Linked List
   - ii. At the end of Linked List
   - iii. At a given position in Linked List

4. **Searching data from Linked List**

i. It is performed in order to find the location of a particular element in the list.

ii. It needs traversing through the list and make the comparis of every element of the list with the specified elemen

IX. Flow Chart (Attach separate paper if required)



X. 'C' Program Code(Attach separate paper if required

2. No. of pointers to be manipulated in a linked list to insert an item in the middle ...
   a. Two b. Three c. One d. Zero
3. Linked lists are not suitable for data structures of which one of the following problem?
   a. Insertion sort b. Binary search c. Radix sort d. Polynomial manipulation problem
4. The linked list field(s) are
   a. Data b. Pointer c. Pointer to next node d. Data and pointer to next node

(Space for Answer)

Practical No. 11: Perform operation on circular singly linked list

I. Practical Significance

In some applications Dynamic memory allocation using simple singly Linked List may prone to storage wastage if data items are deleted. So simple singly Linked List can be used in circular manner. It gives the flexibility in storage management.

II. Relevant Program Outcomes (POs)

- **Basic knowledge:** Apply knowledge of basic mathematics, sciences and basic engineering to solve the broad-based Computer engineering problem.
- **Discipline knowledge:** Apply Information Technology knowledge to solve broad- based Information Technology related problems.
- **Experiments and practice:** Plan to perform experiments, practices and to use the results to solve Information Technology related problems.
- **Engineering tools:** Apply appropriate Computer Engineering / Information Technology related techniques/ tools with an understanding of the limitations.
- **Communication:** Communicate effectively in oral and written form.

III. Competency and Practical skills

This practical is expect to develop the following skills in you

Develop 'C' programs to solve broad-based computer group related problems.

1. Write algorithm and draw Flow Chart for inserting and deleting nodes in circular singly linked list.
2. Write C program for inserting and deleting nodes in circular singly linked list.
3. Compile/Debug/ Save C program for inserting and deleting nodes in circular singly linked list.

IV. Relevant Course Outcome(s)

- Implement basic operations on Linked List.

V. Practical Outcome (PrOs)

- Write C program to perform the operations (Insert, Delete, Traverse and Search) on Circular singly linked list.

VI. Relevant Affective domain related Outcome(s)

1. Follow ethical practices.
2. Follow safety practices.

XI. Resources required

Sr. No.	Name of Resource	Specification	Quantity	Remarks
1	Hardware: Computer System	Computer (i3-i5 preferable), RAM minimum 2 GB and onwards	As per batch size	For all Experiments
2	Operating system	Windows 7 or Later Version/LINUX version 5.0 or Later Version		
3	Software	Turbo C /C++ Version 3.0 or later		

XII. Precautions

1. Be careful while giving input for dynamic memory allocation.
2. Follow pointer's fundamentals.

XIII. Resources used

S. No.	Name of Resource	Specification
1	Computer System with broad specifications	Computer (i3-i5 preferable) RAM minimum 2 GB and onwards
2	Software	Turbo c/c++ version 3.0 or later
3	Any other resource used	Windows 7 or later version

XIV. Result

.....

.....

.....

.....

XV. Conclusion(s)

We have performed this practicle to perform operation on singly linked list successfully

.....

.....

.....

XVI. Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

Choose right option for following:

1. Single linked list uses _____ no. of pointers

a. Zero **b.** one **c.** Two **d.** Three

VII. Minimum Theoretical Background

Circular Linked List: Circular Linked List is a variation of Linked list in which the first element points to the last element and the last element points to the first element. Both Singly Linked List and Doubly Linked List can be made into a circular linked list.

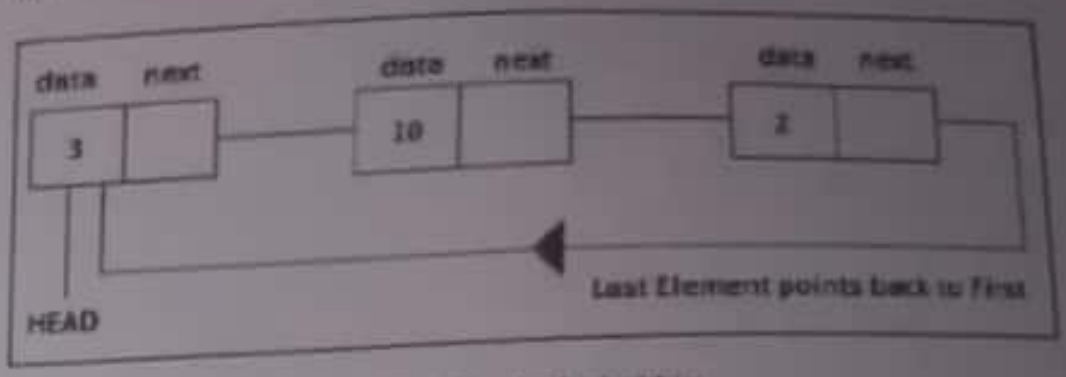


Fig 1.Circular Linked List

Head is a pointer variable of type struct Node which acts as the Head to the list. Initially Head is connected to last node of list. The operations we can perform on Circular singly linked lists are insertion, deletion and traversal.

VIII. Algorithm (Attach separate paper if required)

1. **Insert a node**
   - iv. At Beginning of Linked List
   - v. At the end of Linked List
   - vi. At a given position in Linked List

2. **Delete a node**
   - iv. At Beginning of Linked List
   - v. At the end of Linked List
   - vi. At a given position in Linked List

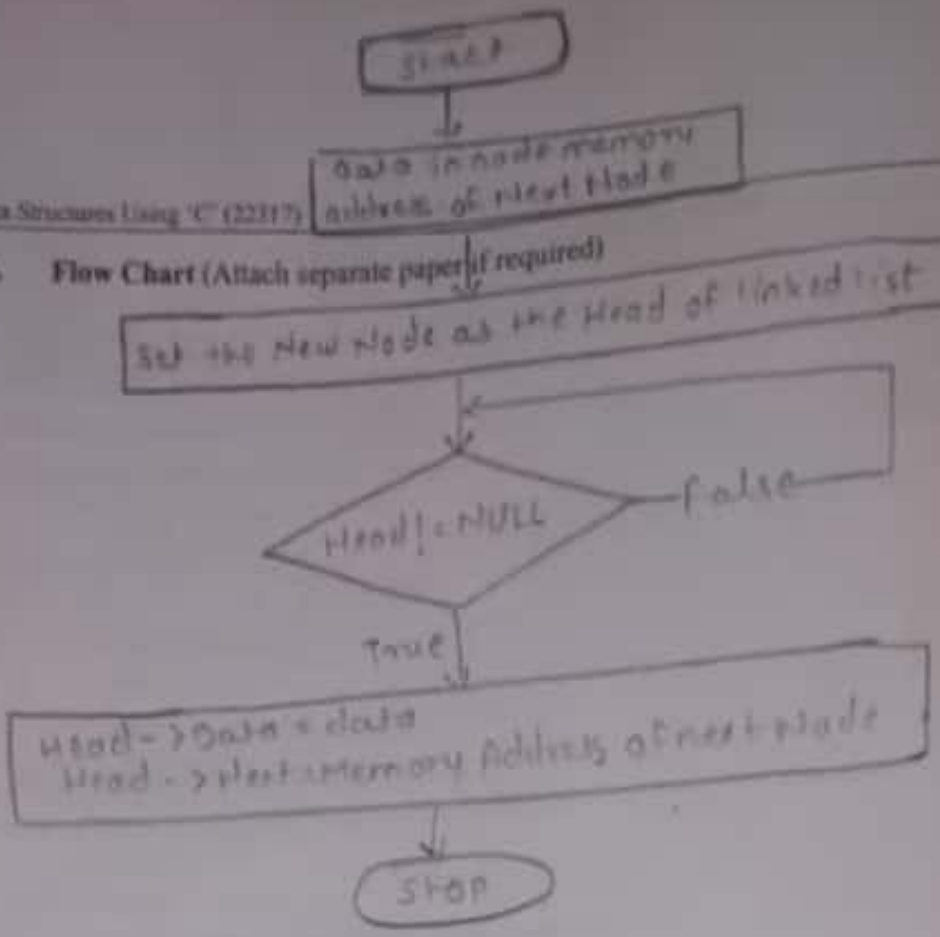
3. **Traversing Linked List**

i. It can be done through a loop.
ii. Initialize the temporary pointer variable temp to head
iii. Run the while loop until the next pointer of temp becomes

4. **Searching data from Linked List**

i. It needs traversing across the list.
ii. The item which is to be searched in the list is matched each node data of the list once.
iii. If the match found then the location of that item is returned otherwise -1 is returned.

IX. Flow Chart (Attach separate paper if required)



X. 'C' Program Code (Attach separate paper if required)

```
#include <stdio.h>
#include <stdlib.h>
struct node {
int data;
struct node *next;
} *head;
void createList(int n);
void displayList();
void insert_beginning(int data);
void search_element(int data);
void updating_element(int data);
void insert_given_position(int data, int position);
void delete_beginning();
void delete_given_position();
void reverse_list();
int main()
{int n, data, choice = 1;
 head = NULL;
 while(choice != 0)
 {
```

```
printf("\n\n\t CIRCULAR LINKED LIST BASIC OPERATIONS \n\n");
printf("1. Create list\n");
printf("2. Insert at beginning\n");
printf("3. Insert at any position\n");
printf("4. Delete at beginning\n");
printf("5. Delete at any position\n");
printf("6. Search Element\n");
printf("7 Update Element\n");
printf("8 - Reverse List\n");
printf("0 - Exit\n");
printf("\n\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch(choice)
{
case 1:
printf("Enter the total number of nodes in list: ");
scanf("%d", &n);
createlist(n);
displaylist();
break;
case 2:
printf("Enter data tobe inserted at beginning: ");
scanf("%d", &data);
insert_beginning(data);
displaylist();
break;
case 3:
printf("Enter node position: ");
scanf("%d", &n);
printf("Enter data you want to insert at %d position:", n);
scanf("%d", &data);
insert_given_position(data, n);
displaylist();
break;
case 4:
if(head==NULL)
{
printf("\n The list is empty\n");
}
else
{
delete_beginning();
displaylist();
}
break;
case 5:
if(head==NULL)
```

```
printf("\n\n\t CIRCULAR LINKED LIST BASIC OPERATIONS:\n\n");
printf("1. Create list\n");
printf("2. Insert at beginning\n");
printf("3. Insert at any position\n");
printf("4. Delete at beginning\n");
printf("5. Delete at any position\n");
printf("6. Search Element\n");
printf("7. Update Element\n");
printf("8. Reverse list\n");
printf("0. Exit\n");
printf("\n\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch(choice)
{
case 1:
printf("Enter the total number of nodes in list: ");
scanf("%d", &n);
createlist(n);
displaylist();
break;
case 2:
printf("Enter data to be inserted at beginning: ");
scanf("%d", &data);
insert_beginning(data);
displaylist();
break;
case 3:
printf("Enter node position: ");
scanf("%d", &n);
printf("Enter data you want to insert at %d position:", n);
scanf("%d", &data);
insert_given_position(data, n);
displaylist();
break;
case 4:
if(head == NULL)
{
printf("\n The list is empty\n");
}
else
{
delete_beginning();
displaylist();
}
break;
case 5:
if(head == NULL)
{
printf("\n The list is empty\n");
}
else
{
delete_given_position();
displaylist();
```

```
newNode->next = current->next;
current->next = newNode;
}
void delete_beginning()
{
struct node *temp, *s;
if (head == head->next)
{
head=NULL;
printf("\n The List is empty\n");
}
else {
temp=head;
s = head;
while (temp->next != head)
{
temp=temp->next;
}
printf("\nThe element %d is deleted at the beginning", s->data);
printf("\n");
head = s->next;
temp->next = head;
printf("\n");
free(s);
}
}
void delete_given_position()
{
struct node *temp, *s;
if (head==NULL)
printf("\n The List is empty");
else {
int count=0, pos;
printf("\nEnter the position to be deleted: ");
scanf("%d", &pos);
temp = head;
while (count < pos)
{
s=temp;
temp=temp->next;
count++;
}
printf("\n The element %d at index %d is deleted", temp->data, pos);
printf("\n");
s->next = temp->next;
printf("\n");
free(temp);
}}
void search_element(int data)
{
struct node *temp=head;
```

```
while(temp)
{
if(temp->data==data)
{
printf("\n Element found at index %d in the list ",index);
break; }
else{
temp=temp->next;
index++;
}
} }
void updating_element (int data)
{
int new_data;
printf("\n Enter the new data to replace with: ");
scanf("%d",&new_data);
struct node *temp=head;
while(temp)
{
if(temp->data==data)
{
temp->data=new_data;
break; }
else{
temp=temp->next;
}
}
}
void reverse_list()
{ struct node *prev,*cur,*next,*last;
printf("\n The reversed list is\n\n");
if(head==NULL)
{
printf("cannot reverse empty list.\n");
return;
}
last=head;
prev=head;
cur=(head)->next;
head=(head)->next;
while(head!=last)
{
head=head->next;
cur->next=prev;
prev=cur;
cur=head;
}
cur->next=prev;
head=prev;
display_list();
}
```

```
while(temp)
{
if(temp->data==data)
{
printf("\n Element found at index %d in the list ",index);
break; }
else{
temp=temp->next;
index++;
}
} }
void updating_element (int data)
{
int new_data;
printf("\n Enter the new data to replace with: ");
scanf("%d",&new_data);
struct node *temp=head;
while(temp)
{
if(temp->data==data)
{
temp->data=new_data;
break; }
else {
temp=temp->next;
}
}
}
void reverse_list()
{ struct node *prev,*cur,*next,*last;
printf("\n The reversed list is\n\n");
if(head==NULL)
{
printf("Cannot reverse empty list.\n");
return;
}
last=head;
prev=head;
cur=(head)->next;
head=(head)->next;
while(head!=last)
{
head=head->next;
cur->next=prev;
prev=cur;
cur=head;
}
cur->next=prev;
head=prev;
display list();
}
```

XI. Resources required

Sr. No.	Name of Resource	Specification	Quantity	Remarks
1	Hardware: Computer System	Computer (i3-i5 preferable), RAM minimum 2 GB and onwards	As per batch size	For all Experiments
2	Operating system	Windows 7 or Later Version/LINUX version 5.0 or Later Version		
3	Software	Turbo C /C++ Version 3.0 or later		

XII. Precautions

1. Be careful while giving input for dynamic memory allocation.
2. Keep track of next pointer to observe Circular singly linked list.

XIII. Resources used

S. No.	Name of Resource	Specification
1	Computer System with broad specifications	Computer (i3-i5 preferable), RAM minimum 2GB and onwards
2	Software	Turbo C/C++ Version 3.0 or later
3	Any other resource used	Windows 7 or later version/LINUX version 5.0 or later version

XIV. Result (Output of the Program)

XV. Conclusion(s)

We have performed this practicle to perform Operation on circular singly linkedlist successfully.

XVI. Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

Choose the right option from following:

1. Circular Single linked list uses _____ no. of pointers

a. Zero b. one c. Two d. Three

2. No. of pointers to be manipulated in a Circular linked list to insert an item in the middle
   a. Two b. Three c. One d. Zero
3. Linked lists are not suitable for data structures of which one of the following problem?
   a. insertion sort b. Binary search c. radix sort
   d. polynomial manipulation problem
4. The last node of Circular linked list field(s) having
   a. data b. pointer c. pointer to next node d. pointer to first node

XVII. Exercise

1. Give the benefit of Circular Single Linked List.
2. Represent Linked List as Circular Queue.

(Space for answers)

1.→① No requirement for a NULL assignment in the code.
② The circular list never points to a NULL pointer unless fully deallocated.
③ Circular linked lists are advantageous for end operations since beginning and end coincide.

Practical No. 12: Perform traversing on binary search tree

I. Practical Significance

To perform operations in some application which works on hierarchical data a Tree data structure is most known Data structure. Tree is used to represent data in hierarchical manner. A binary tree can be used to represent ordered array and a linked list.

II. Relevant Program Outcomes (POs)

- **Basic knowledge:** Apply knowledge of basic mathematics, sciences and basic engineering to solve the broad-based Computer engineering problem.
- **Discipline knowledge:** Apply Information Technology knowledge to solve broad-based Information Technology related problems.
- **Experiments and practice:** Plan to perform experiments, practices and to use the results to solve Information Technology related problems.
- **Engineering tools:** Apply appropriate Computer Engineering / Information Technology related techniques/ tools with an understanding of the limitations.
- **Communication:** Communicate effectively in oral and written form.

III. Competency and Practical skills

This practical is expect to develop the following skills in you

Develop 'C' programs to solve broad-based computer group related problems.

1. Write algorithm and draw Flow Chart for Traversing Tree.
2. Write a C program for Traversing Tree
3. Compile/Debug/Save the C program.

IV. Relevant Course Outcome(s)

- Implement program to create and traverse tree to solve problems.

V. Practical Outcome

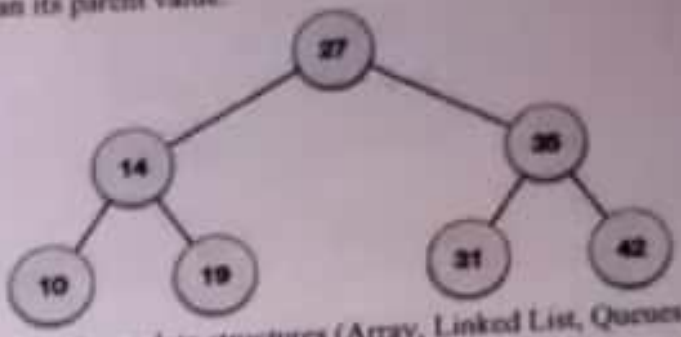
- Write C program to implement BST(Binary Search Tree) and traverse the tree (Inorder, Preorder, Postorder)

VI. Relevant Affective domain related Outcome(s)

1. Follow safety measures
2. Follow ethical practices.

VII. Minimum Theoretical Background

Binary Search Tree (BST) : A binary search tree is a tree where each node has a left and right child. Either child, or both children, may be missing. A node's left child must have a value less than its parent's value and the node's right child must have a value greater than its parent value.



Traversing : Unlike linear data structures (Array, Linked List, Queues, Stacks, etc) which have only one logical way to traverse them, trees can be traversed in different ways. Following are the generally used ways for traversing trees.

Tree Traversals:

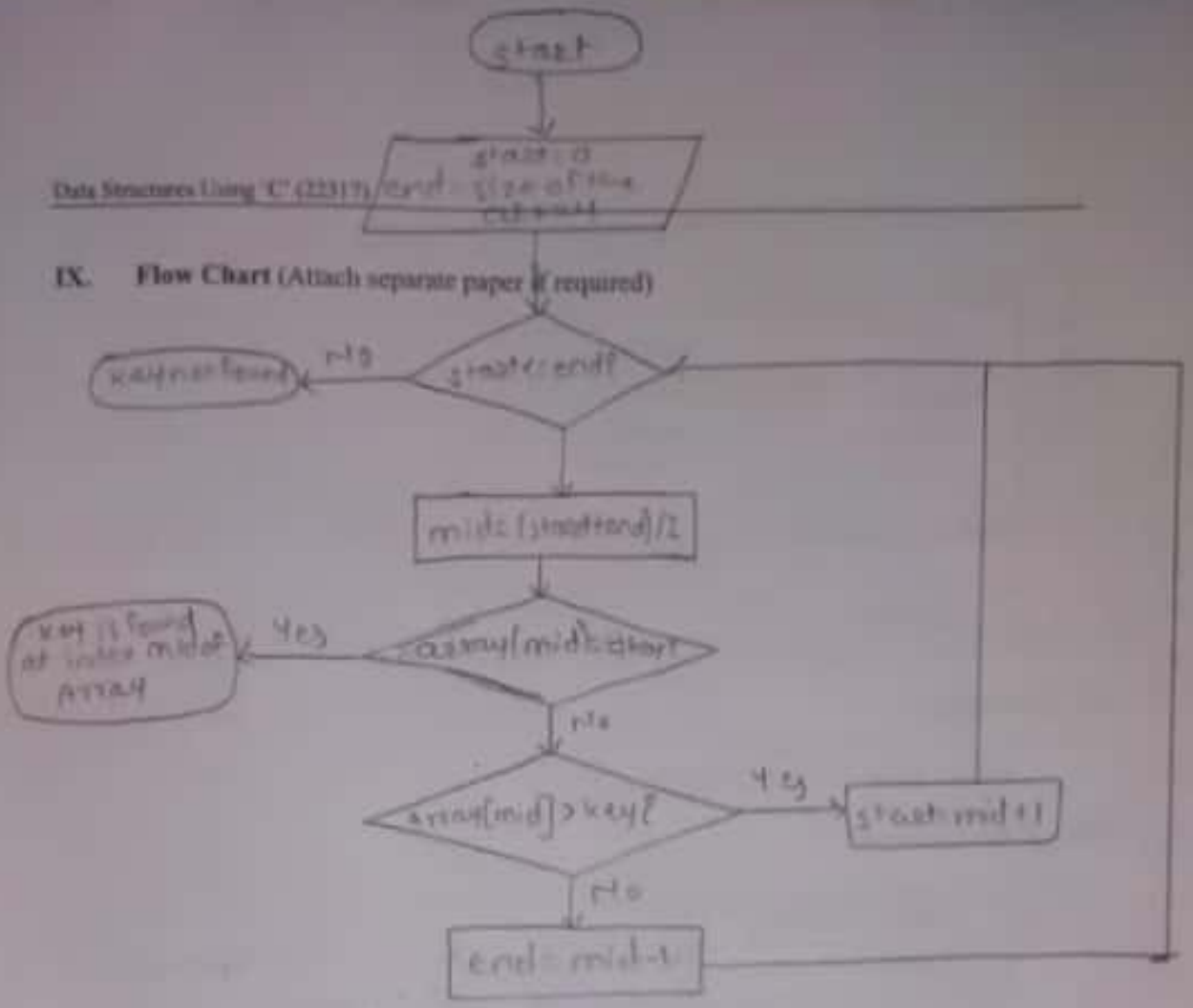
(a) Inorder
(b) Preorder
(c) Postorder

VIII. Algorithm (Attach separate paper if required)

1. Create Binary Search Tree

```
struct node* search (int data){
  struct node* current = root;
print("Visiting elements: ");
while(current-> data != data){
  if(current != NULL){
print f("%d", & current-> data);
if (current-> data > data){
  current = current-> left child;
}
else{
  current = current-> right child;
}
if(current == NULL){
  return NULL;
}
}
return current;
```

IX. **Flow Chart** (Attach separate paper if required)



X. **'C' Program Code**(Attach separate paper if required)

```
#include<stdio.h>
#include<stdlib.h>
struct btnode
{
int value;
struct btnode *l;
struct btnode *r;
}*root=NULL,*temp=NULL,*t2,*t1;
void delete1();
void insert();
void delete();
void inorder(struct btnode *t)
void create();
void search(struct btnode *t);
void preorder(struct btnode* t);
void postorder(struct btnode *t);
void search1(struct btnode *t,int data);
int smallest(struct btnode *t);
int largest(struct btnode *t);
```

```
flag=1;
void main(){
int ch;
printf("\nOPERATIONS...");
printf("\n1-Insert an element into tree\n");
printf("\n2-Delete an element from the tree\n");
printf("3-Inorder Traversal\n");
printf("4-Preorder Traversal\n");
printf("5-Postorder Traversal\n");
printf("6-Exit\n");
while(1)
{
printf("\nEnter your choice:");
scanf("%d",&ch);
switch(ch)
{
case 1:
insert();
break;
case 2:
delete();
break();
case 3:
inorder(root);
break();
case 4:
preorder(root);
break();
case 5:
postorder(root);
break;
case 6:
exit(0);
default:
print("Wrong choice, Please enter correct choice");
break;
}
}
}
/* To insert a node in the tree */
void insert()
{
create();
if(root==NULL)
root=temp;
else
search(root);
}
/* To create a node */
void create()
{ int data;
printf("Enter data of node to be inserted:");
scanf("%d",&data);
temp=(struct btnode*)malloc(1*sizeof(struct btnode));
temp->value=data;
temp->l=temp->r=NULL;
```

```
..... */
/* Function to search the appropriate position to insert the newnode */
void search (struct btnode *t)
{
if ((temp->value > t->value) && (t->r != NULL))
search (t->r)
else if ((temp->value > t->value) && (t->l != NULL))
t->r = temp;
else if ((temp->value < t->value) && (t->l != NULL))
search (t->l);
else if ((temp->value < t->value) && (t->l == NULL))
t->l = temp;
}
/* recursive function to perform inorder traversal of tree */
void inorder (struct btnode *t)
{
if (root == NULL)
{
printf ("No elements in a tree to display");
return;
}
if (t->l != NULL)
inorder (t->l)
printf ("%d->", t->value);
if (t->r != NULL)
inorder (t->r);
}
/* To check for the deleted node */
void delete()
{
int data;
if (root == NULL)
{
printf ("No elements in a tree to delete");
return;
}
printf ("Enter the data to be deleted:");
scanf ("%d", &data);
t1 = root;
t2 = root;
search ((root, data);
}
/* To find the preorder traversal */
void preorder (struct btnode *t)
{
if (root == NULL)
{
printf ("No elements in a tree to display");
return;
}
printf ("%d->", t->value);
if (t->l != NULL)
preorder (t->l)
if (t->r != NULL)
preorder (t->r);
```

```
/* To find the postorder traversal */
void postorder (struct btnode *t)
{
if(root ==NULL)
{
printf("No elements in a tree to display");
return;
}
if (t->l !=NULL)
 postorder (t->l);
if(t->r !=NULL)
 postorder (t->r);
printf ("%d->", t->value);
}
/*Search for the appropriate position to insert the new node*/
void search1 (struct btnode *t, int data)
{if ((data>t->value))
{
 t1 = t;
 search1 (t->l, data);
}
else if ((data==t->value))
{delete1(t);
}
}
/*To delete a node*/
void delete1 (struct btnode *t)
{
 int k;
 /*To delete leaf node*/
if ((t->l == NULL) && (t->r == NULL))
{
 if (t1->l == t)
 {
 t1->l =NULL;
 }
 else
 {t1->r =NULL;
 }
 t = NULL;
 free(t);
 return;
}
/*To delete node having one left hand child */
else if ((t->r == NULL))
{if (t1 == t)
{
root = t->l;
t1 =root;
}
else if (t1->l == t)
{
```